EN.553.740 Machine Learning 1

Jonathan Ma

October 23, 2026

Contents

0.1 Introduction . . . 2
0.1.1 Learning Problems . . . 2
0.2 Mathematical Review . . . 3
0.2.1 Linear Algebra . . . 3
0.2.2 Calculus . . . 9
0.2.3 Probability . . . 16
0.2.4 Matrix Analysis . . . 21
0.3 Optimization and Unconstrained Optimization . . . 26
0.3.1 Basic Optimization Terminology . . . 26
0.3.2 First-Order Necessary Condition . . . 27
0.3.3 Second-Order Optimality Conditions . . . 28
0.3.4 Convexity and Global Optimality . . . 29
0.3.5 Gradient-Based Methods . . . 30
0.3.6 Stochastic Gradient Methods . . . 31
0.4 Introduction to Learning and Kernel Density Estimation . . . 33
0.4.1 Kernels and Bandwidth . . . 33
0.4.2 Kernel Density Estimation . . . 34
0.4.3 Pointwise Error and Bias–Variance . . . 35
0.4.4 Bias and Variance Bounds for KDE . . . 36
0.4.5 Bandwidth Rate and the Curse of Dimensionality . . . 37
0.5 Prediction Basics and the Bayes Predictor . . . 38
0.5.1 Loss Functions . . . 38
0.5.2 Expected Risk . . . 39
0.5.3 Conditional Risk and the Pointwise Rule . . . 39
0.5.4 Bayes Regression under Squared Loss . . . 40
0.5.5 Bayes Classification under Zero–One Loss . . . 40
0.5.6 From the Bayes Rule to a Learning Algorithm . . . 41
0.5.7 Joint Gaussian Model for Regression . . . 41
0.5.8 Naive Bayes . . . 42
0.5.9 Kernel Regression from a Joint Density Estimate . . . 43
0.5.10 The Empirical Distribution and Memorization . . . 43
0.5.11 Empirical Risk Minimization . . . 44
0.5.12 Why the Function Class Matters . . . 45
0.5.13 Regularized ERM . . . 45
0.5.14 Training Error Is Not Test Error . . . 45
0.5.15 Independent Test and Validation Sets . . . 46

0.5.16 Cross-Validation 46
0.5.17 Cross-Validation for Hyperparameter Selection 47

0.1 Introduction

A good way to think of machine learning is instead of a computer taking data and a program to produce output, it takes data and output and attempts to produce a program, ie "learn" a function.

We use it for tasks that are too complex to program, like driving or large scale data analysis. A bit of history of "learning":

- 1959: Arthur Samuel (IBM) created a program to play checkers, introducing rule-based learning.
- 1960: Frank Rosenblatt introduced a linear classifier based on brain neurons, called a Perceptron.
- 1969: Marvin Minsky showed limitations of the perceptron, halting development for 20 years
- 1980: First workshop in machine learning
- 1986: Multilayer networks and backpropagation introduced
- 1990: Vapnik and Chervonenkis introduce kernels and SVM, and termed "Statistical Learning"
- 2002: Rise of graphical models
- 2010: Deep learning takes over
- 2016: ML hits record size with NIPS and ICML doubling in size

Machine learning has been used for a variety of tasks, like stock market prediction, decoding thoughts from brain scans, and self-driving cars.

0.1.1 Learning Problems

Statistical Learning falls under five rough fields:

- Classification (Assign a category)
- Regression (predict a real value)
- Ranking (order items)
- Clustering (item partitioning, like image segmentation)
- Dimensionality Reduction/Manifold Learning (Computer vision)

In a broader sense, there is supervised learning (Classification/regression) and unsupervised learning (Clustering, dimensionality reduction, density estimation) and things like semi-supervised learning, active learning, reinforcement learning, etc.

Supervised Learning For a given problem, we learn a function $f : X \to Y$ such that $x \in \mathcal{X}, \quad y \in \mathcal{Y}$. Classification deals with discrete labels while regression deals with continuous functions.

Unsupervised Learning For a given problem, we learn a function $f : X \to Y$ such that $x \in \mathcal{X}, \quad f(x) \in \mathbb{R}$. Usually embedding (for text), clustering, or density estimation.

0.2 Mathematical Review

Algorithms are built from sets, functions, vectors, matrices, norms, and derivatives, so we introduce common notation here. Consider a student success dataset with the target being quiz score, and features as study time and prior score.

0.2.1 Linear Algebra

0.2.1.1 Sets and Power Sets

Let $A = \{1, 2, 3, 4, 5\}$ be the set of students in the toy dataset. The *power set* is $\mathcal{P}(A) = \{B : B \subseteq A\}$. If $|A| < \infty$, then $|\mathcal{P}(A)| = 2^{|A|}$. A subset $B \subseteq A$ may represent a mini-batch, validation set, or a selected group of samples.

0.2.1.2 Functions

For sets A, B, write $B^A = \{f : A \to B\}$. if $B = \mathbb{R}$, then $\mathbb{R}^A$ is the set of real-valued functions on A. If $|A| < \infty$, then $\mathbb{R}^A \simeq \mathbb{R}^{|A|}$ as vector spaces. For our toy example:

$$x_1 : A \to \mathbb{R} \qquad x_2 : A \to \mathbb{R} \qquad y : A \to \mathbb{R}$$

For a subset $C \subseteq A$, define an *indicator function*:

$$\mathbb{1}_C : A \to \{0, 1\}, \quad \mathbb{1}_C(i) = \begin{cases} 1, i \in C, \\ 0, i \notin C \end{cases}$$

Indicators are used to encode events, groups, labels, masks, etc.

0.2.1.3 Vectors

We use a column convention for vectors. For each student i, write the feature vector as:

$$x_i = \begin{bmatrix} x_i^{(1)} \\ x_i^{(2)} \end{bmatrix} \in \mathbb{R}^2$$

Standard vectors that are useful include the zero vector (0_d), unit vector ($\mathbb{1}_d$), and identity matrix ($I_{\mathbb{R}^d}$):

$$0_d = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix} \quad \mathbb{1}_d = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix} \quad I_{\mathbb{R}^d} = \begin{bmatrix} 1 \cdots 0 \\ \ddots \\ 0 \cdots 1 \end{bmatrix}$$

0.2.1.4 ℓ^p norms

For $p \geq 1$ and $x \in \mathbb{R}^d$, a p-norm is defined as

$$\|x\|_p = (|x^{(1)}|^p + \cdots + |x^{(d)}|^p)^{\frac{1}{p}}$$

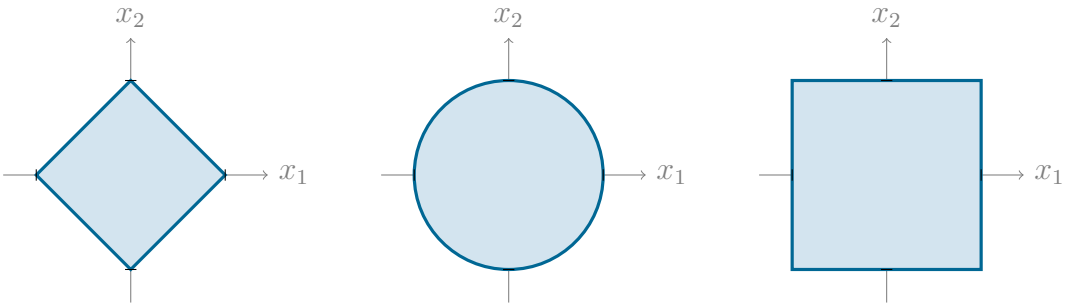
. Special cases include:

$$\ell_1 = \sum_j |x^{(j)}| \quad \ell_2 = \sqrt{\sum_j (x^{(j)})^2} \quad \ell_\infty = \max_j |x^{(j)}|$$

ℓ^2 is called the euclidean norm. We require $p \geq 1$ because the triangle inequality holding is essential for a norm, and only holds for $p \geq 1$:

$$\|x + y\|_p \leq \|x\|_p + \|y\|_p$$
$$\text{when } p = 1; |x| + |y| \leq 1$$
$$\text{when } p = \infty; \max\{|x|, |y|\} \leq 1 \iff |x| \leq 1,$$

A good visualization of norms is shown below:



$\|x\|_1 \leq 1$ $\|x\|_2 \leq 1$ $\|x\|_\infty \leq 1$

Figure 1: Unit balls in $\mathbb{R}^2$. Their boundaries consist of vectors with norm equal to one.

Operator Norms For $A \in \mathcal{M}_{m,d}$, $\|A\|_{op} := \max\{\|Ax\|_2 : x \in \mathbb{R}^d, \|x\|_2 = 1\}$. $\|A\|_{op}$ is the largest stretching factor of the linear map $x \mapsto Ax$. Alternatively, we can derive a similar definition:

$$\|Ax\|_2^2 = \langle Ax, Ax \rangle$$
$$\langle Ax, Ax \rangle = (Ax)^T Ax = x^T A^T Ax$$
$$\|Ax\|_{op}^2 = \max_{\|x\|_2 = 1} x^T A^T Ax$$

Operator norms quantify how much perurbations can be amplified.

0.2.1.5 Cauchy-Schwarz Inequality

For $x, y \in \mathbb{R}^d$, $|\langle x, y\rangle| \leq \|x\|_p \|y\|_q$ with equality iff $x = \alpha y$ for some scalar α and $\frac{1}{p} + \frac{1}{q} = 1$.

Proof sketch: For any $\alpha \in \mathbb{R}$,

$$0 \leq \|x - \alpha y\|_2^2 = \|x\|_2^2 - 2\alpha \langle x, y\rangle + \alpha^2 \|y\|_2^2$$

Choosing the best α gives the inequality:

$$\begin{aligned} &\forall \alpha \in \mathbb{R}, \quad \|x - \alpha y\|_2^2 \geq 0 \\ \iff &\langle x - \alpha y, x - \alpha y\rangle \geq 0 \\ \iff &\langle x, x\rangle - 2\alpha \langle x, y\rangle + \alpha^2 \langle y, y\rangle \geq 0 \\ \iff &\|x\|_2^2 - 2\alpha \langle x, y\rangle + \alpha^2 \|y\|_2^2 \geq 0 \end{aligned}$$

Without loss of generality, assume $\|y\|_2 \geq 0$ and take $\alpha = \frac{\|x\|_2}{\|y\|_2}$:

$$\implies \|x\|_2^2 - \frac{2\langle x, y\rangle \|x\|_2}{\|y\|_2} + \|x\|_2 \geq 0$$

$$\|x\|_2 \|y\|_2 \geq \langle x, y\rangle$$

Equality means $x - \alpha^* y = 0$ so $x = \alpha^* y$.

Cauchy-Schwarz is used in ML to control linear scores, since most models are about estimating weights. A linear model estimates $f_w(x) = \langle w, x\rangle$. Cauchy-Schwarz gives

$$|f_w(x)| = |\langle w, x\rangle| \leq \|w\|_2 \|x\|_2$$

This is saying if features have a bounded norm, and weights have a bounded norm, scores cannot explode. This is why norm constraints and regularization appear throughout learning problems.

The inner product cannot be larger than length times length

We now look at a particle sensor example. Suppose independent sensors $X_1, X_2, \cdots, X_n$ estimate the same mean μ:

$$\mathbb{E}(X_i) = \mu \qquad \mathrm{Var}(X_i) = \sigma_i^2$$

Among weighted averages

$$\hat{\mu} = \sum_{i=1}^{n} w_i X_i$$

which weights should we use, keeping $\hat{\mu}$ unbiased? Compute:

$$\mathbb{E}(\hat{\mu}) = \mathbb{E}[\sum_i w_i X_i] = \sum_i w_i \mathbb{E} X_i = \sum_i w_i \mu = \mu$$

Thus

$$\mathbb{E}(\hat{\mu}) = \mu \iff \sum_i w_i = 1$$

Because X_i are independent,

$$\text{Var}(\hat{\mu}) = \text{Var}(\sum_i w_i X_i) = \sum_i w_i^2 \text{Var}(X_i) = \sum_i w_i^2 \sigma_i^2$$

The optimization problem we obtain is therefor minimize $\text{Var}(\hat{\mu})$ s.t. $\sum_i w_i = 1$ since the sum of weights is one,

$$1 = (\sum_i w_i)^2 = (\sum_i (w_i \sigma_i) \frac{1}{\sigma_i})^2$$

Now by Cauchy-Schwarz,

$$1 = (\sum_i w_i)^2 \leq (\sum_i w_i^2 \sigma_i^2)(\sum_i \frac{1}{\sigma_i^2})$$

Recall $|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2$. Therefore

$$\text{Var}(\hat{\mu}) = \sum_i w_i^2 \sigma_i^2 \geq (\sum_i \sigma_i^{-2})^{-1}$$

This makes it so the elements of X and y are $w_i \sigma_i$ and $1/\sigma_i$ respectively.

Equality in Cauchy-Schwarz holds when

$$w_i \sigma_i \propto \sigma_i^{-1} \implies w_i \propto \frac{1}{\sigma_i^2}$$

so we should normalize weights so they sum to one:

$$\frac{\frac{1}{\sigma_i^2}}{\sum_{j=1}^n \frac{1}{\sigma_i^2}}$$

meaning lower variance sensor implies larger weight.

0.2.1.6 Sparsity and Lasso

Sparsity is roughly defined as what happens when $p \to 0$. For a nonzero scalar a:

$$\lim_{p \to 0} \sum_j \left| x^{(j)} \right|^p = \sum_j \lim_{p \to 0} \left| x^{(j)} \right|^p \therefore \lim_{p \to 0} |a|^p = 1 (\text{ imagine } \begin{cases} 0, x^{(j)} = 0 \\ 1, x^{(j)} \neq 0 \end{cases})$$

For $a = 0, |0|^p = 0$. Therefore informally,

$$\lim_{p \to 0} \sum_j \left| x^{(j)} \right|^p = \{ j : x^{(j)} \neq 0 \} = \|x\|_0$$

The quantity $\|x\|_0$ counts the number of nonzero entries, but it is not a norm and optimization with it is often difficult. Common times in ML, we replace $\|x\|_0$ with $\|x\|_1$ to encourage sparsity while keeping convexity

0.2.1.7 Matrices

In ML, rows often represent samples and columns represent features. we say $\mathcal{M}_{m,d} = \mathcal{M}_{m,d}(\mathbb{R}) = \{m \times d \text{ real matrices }\}$. Additionally $\mathcal{M}_d = \mathcal{M}_{d,d}$. Notation may differ, so remember its always (across, down) ie (i,j)/(r,c)/(m,d)

By this definition, a linear prediction vector is $X_w = [\langle x_1, w\rangle, \langle x_2, w\rangle, \cdots, \langle x_i, w\rangle]^T$.

Symmetric and Positive Semidefinite Matrices A symmetric matrix is defined as

$$S_d = \{A \in \mathcal{M}_d : A = A^T\}$$

A positive semidefinite matrix (PSD) is defined as

$$S_d^+ = \{A \in S_d : x^T A x \geq 0 \quad \forall \quad x \in \mathbb{R}^d\}$$

These usually appear as covariance and correlation matrices, and Hessians of convex quadratic objective functions. A positive definite matrix is defined as

$$S_d^{++} = \{A \in S_d : x^T A x > 0 \quad \forall \quad x \neq 0\}$$

This usually comes up when doing ridge regressions and finding hessians with a parameter greater than 0.

Bilinear Forms A bilinear form is a map $a : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ that is linear in each variable. If a is symmetric (ie $a(x,y) = a(y,x)$) then $a(x,y) = x^T A y$ for some symmetric matrix $A \in S_d$.

Inner products are PSD:

$$a(x,y) = \langle x, y\rangle = x^T y$$
$$a(x,y) = \langle x, y\rangle = x^T I_d y$$
$$a(x,x) = x^T x = \|x\|_2^2 \geq 0$$

Orthogonal Matrices An Orthogonal transformation matrix O is defined as

$$O_{m,d} = \{A \in \mathcal{M}_{m,d} : AA^T = I_m\}$$

and for $Q \in O_d, Q^T Q = QQ^T = I_d$. The key idea is orthogonal transformations rpeserve euclidean geometry:

$$\|Qx\|_2^2 = (Qx)^T(Qx) = x^T Q^T Q x = x^T x = \|x\|_2^2$$

Similarly

$$\langle Qx, Qy\rangle = x^T Q^T Q y = innerxy$$

Rotations preserve distances and dot products, which matter for PCA, kernels, and preprocessing

0.2.1.8 Open and Closed Balls

We introduce some basic topology. For $x \in \mathbb{R}^d, r > 0$: A ball is open if

$$B(x,r) = \{y \in \mathbb{R}^d : \|y - x\|_2 < r\}$$

A ball includes the boundary if

$$\bar{B}(x,r) = \{y \in \mathbb{R}^d : \|y - x\|_2 \leq r\}$$

A set $U \subset \mathbb{R}^d$ is open if

$$\forall x \in U, \qquad \exists r > 0, \qquad B(x,r) \subset U$$

A set is closed if its complement is open. The intuition is that inside an open set, every point has a small safety ball around it.

For $A \subset \mathbb{R}^d$, the interior ($\int(A)$) is the largest open set containing A, and the closure $cl(a)$ is the smallest closed set containing A. We can also think of this as the closure iincluding the boundary whereas the interior doesn't.

Compactness : A set $K \subset \mathbb{R}^d$ is compact if every sequence of points in K contains a subsequence that converges to a point in K. Take for instance (0,1) vs [0,1]: The requence $x_n = n^{-1}$ lies in (0,1) and converges to 0, but $0 \notin (0,1)$, so (0,1) is not compact. For [0,1], limits of convergent subsequences stay inside the set.

Heine-Borel Theorem A finite-dimensional Euclidean space:

$$K \subset \mathbb{R}^d \text{ compact} \iff K \text{ is closed and bounded.}$$

Bounded means $\exists R > 0$ s.t. $K \subset B(0,R)$. This is special to finite dimensional euclidean spaces, not other spaces.

Weierstrass Extreme Value Theorem If $K \subset \mathbb{R}^d$ is compact and $f : K \to \mathbb{R}$ is continuous, then f attains its minimum and maximum on K. Put otherways,

$$\exists x_{min}, x_{max} \in K \text{ s.t. } f(x_{min}) = \min_{x \in K} f(x), \qquad f(x_{max}) = \max_{x \in K} f(x)$$

In optimization, we often write $\inf_{w \in K} \ell(w)$. Weierstrass tells us when this minimum is actually achieved. Basically saying that on compact parameter sets, continuous objectives have optimizers. We look at an example:

Look at constrained regression. Consider minimizing least squares over a compact ball

$$K_R = \{w \in \mathbb{R}^d : \|w\|_2 \leq R\}$$

The objective is

$$\ell(w) = \frac{1}{2n} \|Xw - y\|_2^2$$

We can prove this by showing $\ell(w)$ is continuous in w. Then show that K_R is closed and bounded in $\mathbb{R}^d$. By Heine-Borel, K_R is compact, so by Weierstrass, $\exists \hat{w} \in K_R$ s.t. $\ell(\hat{w}) = \min_{w \in K_R} \ell(w)$.

0.2.1.9 Metric Spaces

A metric space is a set $\mathcal{B}$ equipped with a distance $\rho : \mathcal{B} \times \mathcal{B} \to [0.\infty)$ satisfying $\forall x, y, z \in \mathcal{B}$:

$$\rho(x, y) \iff x = y$$

$$\rho(x, y) = \rho(y, x)$$

$$\rho(x, z) \leq \rho(x, y) + \rho(y, z)$$

For $\mathcal{B} \in \mathbb{R}$ and $\rho : \mathbb{R} \times \mathbb{R}$: $\rho(x, y) = |x - y|$.

For $\mathcal{B} \in \mathbb{R}^d$ and $\rho : \mathbb{R}^d \times \mathbb{R}^d$: $\rho(x, y) = \|x - y\|_2$.

Metrics formalize dissimilarity between data points. This could be distances between two feature vectors, probability distributions, or functions/signals. Open, Closed, and compactness still make sense in metric spaces, but Heine-Borel may fail. In $\mathbb{R}$, closed and bounded iff compact, but in a general metric space, it does not imply compactness

0.2.2 Calculus

Calculus Notation Map

Notation	Meaning in these notes
$x, y \in \mathbb{R}$	one-dimensional points, often with $x < y$
$x \in \mathbb{R}^d$	deterministic vector or input point
$e_1, \ldots, e_d$	canonical basis of $\mathbb{R}^d$: $e_1 = (1, 0, ...), e_2 = (0, 1, ...)$
∂f	derivative in one dimension
$\partial_j f$	partial derivative along e_j
$df(x)$	differential or Jacobian matrix at x
$\nabla \cdot f$	divergence, when $f \colon \mathbb{R}^d \to \mathbb{R}^d$
$\nabla^2 f$	Hessian, for scalar-valued f
Δf	Laplacian, $\mathrm{tr}(\nabla^2 f)$

Probability Notation Map

Notation	Meaning in these notes
Ω	sample space
$A \subseteq \Omega$	event
$\mathbb{P}$	probability function or probability measure
$(\Omega, \mathbb{P})$	probability space
X, Y	random variables (capital letters)
x, y	realizations or values of random variables
R_X, R_Y	possible values of X and Y
$[X = x]$	event $\{\omega \in \Omega : X(\omega) = x\}$
$\mathbb{E}[X]$	expectation of X
$\mathbb{E}[Y \mid X = x]$	conditional expectation of Y given $X = x$

A model is often a function $f(x)$ and a loss is often a function $\ell(w)$ of parameters w. Optimization therefore asks how does $\ell(w)$ change after w is perturbed, or, if we move from w to $w + h$, can we predict whether loss goes down?

Intervals Assume $x, y \in \mathbb{R}; x > y$. The line segment from x to y is written by $(1-t)x + ty$. Define the intervals as follows:

$$[x, y] := \{(1-t)x + ty : 0 \leq t \leq 1\}$$

$$[x, y) := \{(1-t)x + ty : 0 \leq t < 1\}$$

$$(x, y] := \{(1-t)x + ty : 0 < t \leq 1\}$$

$$(x, y) := \{(1-t)x + ty : 0 < t < 1\}$$

The same line segment formula will appear in convexity, Jensen's Inequality, and taylor expansion along a direction.

0.2.2.1 1D Derivatives

Let $f : I \subset \mathbb{R} \to \mathbb{R}, \quad t \mapsto f(t) \in \mathbb{R}$. The derivative of f at $t_0 \in I$ is

$$\partial f(t_0) = \partial f|_{t=t_0} := \lim_{h \to 0} \frac{f(t_0 + h) - f(t_0)}{h}$$

when this limit exists. For higher-order derivatives:

$$\partial^k f := \partial(\cdots \partial(\partial f)), \quad k \geq 0, \quad k \in \mathbb{Z}^+$$

with the convention $\partial^0 f := f$. The derivative is the local slope of the function at t_0.

0.2.2.2 Higher-Dimensional Derivatives

Now move to $d > 1$, and let $f : U \subset \mathbb{R}^d \to \mathbb{R}^m$ and U is *open* (meaning we can move a little around x without immediately leaving the domain).

0.2.2.3 Partial Derivates

Let $e_{1,\dots,}e_d$ be the canonical basis of $\mathbb{R}^d$. For $1 \leq j \leq d$:

$$\partial_j f(x) := \partial_t(f(x + te_j))|_{t=0} = \lim_{t\to 0} \frac{f(x + te_j) - f(x)}{t}$$

This means that e_j changes only coordinate j and $x + te_j$ means move from x only along coordinate j. The partial derivative measures local change in that coordinate direction.

For example, let $f(\theta) = f(x, y) = x^2 + 3xy$. Then $\partial_x f(\theta) = 2x + 3y$ and $\partial_y f(x) = 3y$, and at $\theta = (1, 2)^T$, $\partial_x f(\theta) = 8$ and $\partial_y f(x) = 3$

0.2.2.4 Directional Derivatives

Partial derivatives measure change along the coordinate directions. More generally, for a direction $h \in \mathbb{R}^d$, the *directional derivative* of f at x in the direction h is

$$\partial_t f(x + th)|_{t=0}\,.$$

A partial derivative is the special case $h = e_j$, whereas a directional derivative allows any $h \in \mathbb{R}^d$.

In machine learning, if x is an input image or feature vector and h is a small perturbation, the directional derivative describes how the model output changes under that perturbation.

0.2.2.5 The Differential

Let $f\colon U \subset \mathbb{R}^d \to \mathbb{R}^m$, with component functions $f^{(1)}, \dots, f^{(m)}$. The *differential* of f at x is

$$df(x) := \left(\partial_j f^{(i)}(x)\right)_{1\leq i\leq m,\, 1\leq j\leq d}\,.$$

Equivalently,

$$df(x) = \begin{pmatrix} \partial_1 f^{(1)}(x) & \cdots & \partial_d f^{(1)}(x) \\ \vdots & \ddots & \vdots \\ \partial_1 f^{(m)}(x) & \cdots & \partial_d f^{(m)}(x) \end{pmatrix} \in \mathbb{R}^{m\times d}.$$

Thus, row i contains the partial derivatives of the output component $f^{(i)}$, while column j contains the partial derivatives with respect to the input coordinate x_j.

Example 0.2.1 (Computing $df(x)h$)**.** Consider the scalar-valued function

$$f\left(x^{(1)}, x^{(2)}\right) = \left(x^{(1)}\right)^2 + 3x^{(1)}x^{(2)}.$$

Its differential is the row vector

$$df(x) = \begin{pmatrix} \partial_1 f(x) & \partial_2 f(x) \end{pmatrix} = \begin{pmatrix} 2x^{(1)} + 3x^{(2)} & 3x^{(1)} \end{pmatrix}.$$

At $x = (1, 2)^T$,

$$df(x) = \begin{pmatrix} 8 & 3 \end{pmatrix}.$$

Therefore, if $h = (1, -1)^T$, then

$$df(x)h = \begin{pmatrix} 8 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = 5.$$

0.2.2.6 Product Rule

Let $f, g\colon U \to \mathbb{R}$. Then

$$d(fg)(x) = f(x)\,dg(x) + g(x)\,df(x).$$

The product rule says that the first-order change of a product has two contributions: one from changing f and one from changing g.

0.2.2.7 Chain Rule

Let

$$f\colon U \subset \mathbb{R}^d \to \mathbb{R}^m, \qquad g\colon V \subset \mathbb{R}^b \to U.$$

The condition $g(V) \subseteq U$ ensures that $f(g(x))$ is defined for every $x \in V$. The composition satisfies $f \circ g\colon V \to \mathbb{R}^m$, and the chain rule is

$$d(f \circ g)(x) = df\Big(g(x)\Big)\,dg(x).$$

The dimensions are compatible because

$$\underbrace{df\Big(g(x)\Big)}_{m\times d}\,\underbrace{dg(x)}_{d\times b} = \underbrace{d(f\circ g)(x)}_{m\times b}.$$

Modern machine-learning models are compositions of functions:

$$x \longmapsto h_1(x) \longmapsto h_2\Big(h_1(x)\Big) \longmapsto \cdots \longmapsto f(x).$$

The chain rule explains how derivatives propagate through such a composition. In particular, backpropagation in a neural network is repeated application of the chain rule.

0.2.2.8 Divergence

When $d = m$, a function $f\colon U \subset \mathbb{R}^d \to \mathbb{R}^d$ is a vector field. Its *divergence* is

$$\nabla \cdot f(x) := \operatorname{tr}\Big(df(x)\Big).$$

Writing

$$f(x) = \begin{pmatrix} f^{(1)}(x) \\ \vdots \\ f^{(d)}(x) \end{pmatrix},$$

the divergence is the sum of the diagonal entries of the $d \times d$ differential:

$$\nabla \cdot f(x) = \partial_1 f^{(1)}(x) + \cdots + \partial_d f^{(d)}(x).$$

Example 0.2.2 (Computing divergence)**.** Let $f\colon \mathbb{R}^2 \to \mathbb{R}^2$ be

$$f(x) = \begin{pmatrix} \big(x^{(1)}\big)^2 \\ x^{(1)}x^{(2)} \end{pmatrix}.$$

Then

$$df(x) = \begin{pmatrix} 2x^{(1)} & 0 \\ x^{(2)} & x^{(1)} \end{pmatrix}.$$

Therefore,

$$\nabla \cdot f(x) = \operatorname{tr}\Big(df(x)\Big) = 2x^{(1)} + x^{(1)} = 3x^{(1)}.$$

Divergence appears in continuous normalizing flows, probability flows, and differential equations for densities.

0.2.2.9 Determinant and Its Derivatives

Let

$$A = \begin{bmatrix} a_1 & \cdots & a_d \end{bmatrix} \in \mathcal{M}_d,$$

where a_i is the ith column of A. The determinant

$$\det \colon \mathcal{M}_d \to \mathbb{R}$$

can be viewed as a d-linear form in the columns:

$$\delta(a_1, \ldots, a_d) = \det(A).$$

It is characterized by the following properties:

- If $a_i = a_j$ for $i \neq j$, then $\delta(a_1, \ldots, a_d) = 0$.
- Swapping two columns multiplies δ by -1.
- $\delta(e_1, \ldots, e_d) = 1$.

Let a_{ij} denote the entry in row i and column j of A, and let $A^{(i,j)}$ be the matrix obtained by deleting row i and column j. The entrywise derivative of the determinant is

$$\partial_{a_{ij}} \det(A) = (-1)^{i+j} \det\Big(A^{(i,j)}\Big).$$

The quantity $(-1)^{i+j} \det\Big(A^{(i,j)}\Big)$ is the *cofactor* corresponding to the entry a_{ij}.

Proof. Expand the determinant along column j:

$$\det(A) = \sum_{r=1}^{d} (-1)^{r+j} a_{rj} \det\Big(A^{(r,j)}\Big).$$

Differentiating with respect to a_{ij} gives

$$\begin{aligned} \partial_{a_{ij}} \det(A) &= \partial_{a_{ij}} \left[\sum_{r=1}^{d} (-1)^{r+j} a_{rj} \det\Big(A^{(r,j)}\Big) \right] \\ &= (-1)^{i+j} \det\Big(A^{(i,j)}\Big). \end{aligned}$$

Only the term with $r = i$ contains a_{ij} directly. □

0.2.2.10 Higher-Order Derivatives

Higher-order derivatives are defined iteratively from first-order derivatives. For $f\colon U \to \mathbb{R}^m$,

$$\partial_{i_k} \cdots \partial_{i_1} f\colon U \to \mathbb{R}^m,$$

where

$$x \longmapsto \partial_{i_k}\Big(\partial_{i_{k-1}} \cdots \partial_{i_1} f\Big)(x).$$

The order of the indices records the order in which the derivatives are taken. Under suitable smoothness conditions, mixed partial derivatives commute.

0.2.2.11 Hessian

For a scalar-valued function $f\colon U \to \mathbb{R}$, the *Hessian* is

$$\nabla^2 f(x) := \begin{pmatrix} \partial_1\partial_1 f(x) & \cdots & \partial_1\partial_d f(x) \\ \vdots & \ddots & \vdots \\ \partial_d\partial_1 f(x) & \cdots & \partial_d\partial_d f(x) \end{pmatrix}.$$

The Hessian records the second-order curvature of a scalar-valued function.

Example 0.2.3 (Computing a Hessian)**.** Let

$$f\Big(x^{(1)}, x^{(2)}\Big) = \Big(x^{(1)}\Big)^2 + 3x^{(1)}x^{(2)} + 2\Big(x^{(2)}\Big)^2.$$

The first derivatives are

$$\partial_1 f = 2x^{(1)} + 3x^{(2)}, \qquad \partial_2 f = 3x^{(1)} + 4x^{(2)}.$$

Therefore, the Hessian is

$$\nabla^2 f(x) = \begin{pmatrix} 2 & 3 \\ 3 & 4 \end{pmatrix}.$$

0.2.2.12 Laplacian

For a scalar-valued function f, the *Laplacian* is

$$\Delta f := \operatorname{tr}\Big(\nabla^2 f\Big).$$

Equivalently,

$$\Delta f = \partial_1^2 f + \partial_2^2 f + \cdots + \partial_d^2 f,$$

where $\partial_j^2 f := \partial_j \partial_j f$.

The Laplacian appears in diffusion, score models, graph Laplacians, manifold learning, and regularization.

0.2.2.13 Taylor's Theorem

0.2.2.14 One-Dimensional Case

Let $f\colon I \subset \mathbb{R} \to \mathbb{R}$ and suppose $x + h \in I$. The order-k Taylor expansion used in these notes is

$$f(x+h) = f(x) + \partial f(x)h + \frac{1}{2}\partial^2 f(x)h^2 + \cdots + \frac{1}{k!}\partial^k f(x)h^k + o(h^k),$$

where

$$\lim_{h\to 0} \frac{o(h^k)}{|h|^k} = 0.$$

Thus, $o(h^k)$ denotes an error term that is negligible compared with $|h|^k$ as $h \to 0$.

An alternative version gives an explicit remainder. There exists a point $\xi \in [x, x+h]$ such that

$$f(x+h) = f(x) + \partial f(x)h + \cdots + \frac{1}{(k-1)!}\partial^{k-1} f(x)h^{k-1} + \frac{1}{k!}\partial^k f(\xi)h^k.$$

The derivatives through order $k-1$ are evaluated at x, while the final remainder uses $\partial^k f(\xi)$ for some ξ between x and $x+h$.

0.2.2.15 Second-Order Approximation

Taking $k = 2$ in the one-dimensional remainder formula gives

$$f(x+h) = f(x) + \partial f(x)h + \frac{1}{2}\partial^2 f(\xi)h^2, \qquad \xi \in [x, x+h].$$

For a high-dimensional loss, the analogous approximation is

$$L(w+h) \approx L(w) + dL(w)h + \frac{1}{2}h^T \nabla^2 L(w)h.$$

The first-order term suggests a descent direction, while the second-order term controls the error and step size.

0.2.2.16 Gradient Descent from Taylor's Theorem

Take $h = -\eta \nabla L(w)$. Substitution into the second-order approximation gives

$$\begin{aligned} L(w+h) &\approx L(w) + dL(w)h + \frac{1}{2}h^T \nabla^2 L(w)h \\ &= L(w) - \eta \left\|\nabla L(w)\right\|_2^2 + \frac{\eta^2}{2}\nabla L(w)^T \nabla^2 L(w) \nabla L(w). \end{aligned}$$

The term $-\eta \left\|\nabla L(w)\right\|_2^2$ decreases the loss, while the order-η^2 term is the curvature penalty. Choosing η too large can therefore destroy descent.

0.2.3 Probability

0.2.3.1 Sample Spaces and Events

The *sample space* Ω is the set of all possible outcomes, and an *event* is a subset $A \subseteq \Omega$. A probability is a function assigning values to events,

$$\mathbb{P}\colon \mathcal{P}(\Omega) \to [0,1],$$

such that

1. $\mathbb{P}(\Omega) = 1$;
2. $0 \le \mathbb{P}(A) \le 1$ for every $A \subseteq \Omega$;
3. if $A_1, A_2, \ldots \subseteq \Omega$ are pairwise disjoint, then

$$\mathbb{P}\left(\bigcup_i A_i\right) = \sum_i \mathbb{P}(A_i).$$

The pair $(\Omega, \mathbb{P})$ is a *probability space*. Intuitively, the sample space lists all possible outcomes, while an event collects some of those outcomes.

Example 0.2.4 (One die roll)**.** For one roll of a fair die,

$$\Omega = \{1,2,3,4,5,6\}.$$

The event that the outcome is even is

$$A = \{2,4,6\}.$$

Since three of the six outcomes are even,

$$\mathbb{P}(A) = \frac{3}{6} = \frac{1}{2}.$$

0.2.3.2 Random Variables and Realizations

Random variables are functions defined on a probability space. We use capital letters $X, Y, \ldots$ for random variables and lowercase letters $x, y, \ldots$ for their realizations or observed values. The notation

$$[X = x]$$

denotes the event

$$\{\omega \in \Omega : X(\omega) = x\}.$$

In supervised learning, we often regard (X, Y) as a random input–output pair. A dataset

$$(x_1, y_1), \ldots, (x_n, y_n)$$

consists of observed realizations of independent copies of (X, Y). Thus, capital X, Y denote random variables before the data are observed, whereas lowercase x_i, y_i denote observed data points.

0.2.3.3 Inclusion–Exclusion

For events $A_1, A_2 \subseteq \Omega$,

$$\mathbb{P}(A_1 \cup A_2) = \mathbb{P}(A_1) + \mathbb{P}(A_2) - \mathbb{P}(A_1 \cap A_2).$$

The intersection is included once in each of $\mathbb{P}(A_1)$ and $\mathbb{P}(A_2)$, so it must be subtracted once. Induction generalizes this identity to any finite collection of events.

0.2.3.4 Discrete Random Variables

A discrete random variable takes values in a discrete or countable space and has a *probability mass function* (p.m.f.). For a fair coin flip represented by the values 1 and 2,

$$R_X = \{1, 2\}, \qquad \mathbb{P}(X = 1) = \mathbb{P}(X = 2) = \frac{1}{2}.$$

For a Poisson random variable,

$$R_X = \mathbb{N}, \qquad \mathbb{P}(X = i) = \frac{e^{-\lambda}\lambda^i}{i!}, \qquad \lambda > 0.$$

0.2.3.5 Continuous Random Variables

A continuous random variable takes values in a subset of $\mathbb{R}^d$ and has a *probability density function* (p.d.f.). The uniform density on $[0, 1]$ is

$$f(x) = \begin{cases} 1, & x \in [0, 1], \\ 0, & \text{otherwise.} \end{cases}$$

The density of a Gaussian random variable $X \sim \mathcal{N}(\mu, \sigma^2)$ is

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

0.2.3.6 Expectation

Some random variables are neither purely discrete nor purely continuous. For a discrete random variable with p.m.f. f,

$$\mathbb{E}[X] = \sum_{x \in R_X} x f(x).$$

For a continuous random variable with p.d.f. f,

$$\mathbb{E}[X] = \int_{x \in R_X} x f(x)\, \mathrm{d}x.$$

Expectation is a probability-weighted average.

In machine learning, the population risk is an expectation:

$$R(f) = \mathbb{E}\Big[\ell(f(X), Y)\Big].$$

Usually, however, we observe only the data

$$(x_1, y_1), \ldots, (x_n, y_n).$$

We therefore approximate the population risk by the empirical risk

$$\widehat{R}(f) = \frac{1}{n} \sum_{i=1}^{n} \ell\Big(f(x_i), y_i\Big).$$

Empirical risk minimization replaces an unknown expectation with a computable average.

0.2.3.7 Convexity and Jensen's Inequality

A function φ is *convex* if, for every $t \in [0, 1]$ and every x_1, x_2 in its domain,

$$\varphi\Big(tx_1 + (1-t)x_2\Big) \leq t\varphi(x_1) + (1-t)\varphi(x_2).$$

Thus, the graph of a convex function lies below the line segment joining any two points on its graph. For an affine function, expectation commutes with the function:

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b.$$

For a nonlinear function φ, however, $\mathbb{E}[\varphi(X)]$ and $\varphi(\mathbb{E}[X])$ need not be equal.

For convex φ, *Jensen's inequality* states that

$$\varphi\Big(\mathbb{E}[X]\Big) \leq \mathbb{E}[\varphi(X)].$$

The left side first averages X and then applies φ; the right side first applies φ and then averages. Convexity says the latter is at least as large. Jensen's inequality appears in likelihood bounds, variational inference, expectation–maximization, and many generalization and optimization arguments.

Example 0.2.5 (A ratio of identically distributed variables)**.** Suppose $X, Y > 0$ have the same distribution and the relevant expectations exist. Because log is concave, Jensen's inequality applied to X/Y gives

$$\log \mathbb{E}\left[\frac{X}{Y}\right] \geq \mathbb{E}\left[\log \frac{X}{Y}\right].$$

Since X and Y have the same distribution,

$$\mathbb{E}\left[\log \frac{X}{Y}\right] = \mathbb{E}[\log X] - \mathbb{E}[\log Y] = 0.$$

Consequently,

$$\mathbb{E}\left[\frac{X}{Y}\right] \geq 1.$$

This argument does not require X and Y to be independent.

0.2.3.8 Conditional Probability and Bayes' Rule

Let $X\colon \Omega \to R_X$ and $Y\colon \Omega \to R_Y$ be discrete random variables. If $\mathbb{P}(X = x) > 0$, define

$$\mathbb{P}(Y = y \mid X = x) = \frac{\mathbb{P}(Y = y, X = x)}{\mathbb{P}(X = x)}.$$

If $\mathbb{P}(X = x) = 0$, this expression is undefined in the elementary discrete setting.

For a fixed x with $\mathbb{P}(X = x) > 0$, conditional probability is a p.m.f. in y because

$$\begin{aligned}\sum_{y \in R_Y} \mathbb{P}(Y = y \mid X = x) &= \sum_{y \in R_Y} \frac{\mathbb{P}(Y = y, X = x)}{\mathbb{P}(X = x)} \\ &= \frac{\sum_{y \in R_Y} \mathbb{P}(Y = y, X = x)}{\mathbb{P}(X = x)} \\ &= \frac{\mathbb{P}(X = x)}{\mathbb{P}(X = x)} = 1.\end{aligned}$$

In both discrete and continuous settings, Bayes' rule has the form

$$\mathbb{P}(Y \mid X) = \frac{\mathbb{P}(X \mid Y)\mathbb{P}(Y)}{\mathbb{P}(X)},$$

with probabilities replaced by densities when appropriate. In the discrete case,

$$\mathbb{P}(Y = y \mid X = x) = \frac{\mathbb{P}(X = x \mid Y = y)\mathbb{P}(Y = y)}{\mathbb{P}(X = x)}.$$

Bayes classifiers choose the class having the largest posterior probability.

Example 0.2.6 (A diagnostic test)**.** Let D be the event that a person has a rare disease and A the event of a positive test. Suppose

$$\mathbb{P}(D) = 0.01, \qquad \mathbb{P}(A \mid D) = 0.95, \qquad \mathbb{P}(A \mid D^c) = 0.03.$$

Bayes' rule gives

$$\begin{aligned}\mathbb{P}(D \mid A) &= \frac{\mathbb{P}(A \mid D)\mathbb{P}(D)}{\mathbb{P}(A \mid D)\mathbb{P}(D) + \mathbb{P}(A \mid D^c)\mathbb{P}(D^c)} \\ &= \frac{0.95(0.01)}{0.95(0.01) + 0.03(0.99)} \approx 0.242.\end{aligned}$$

0.2.3.9 Conditional Expectation

Since $\mathbb{P}(Y = y \mid X = x)$ is a p.m.f. in y, define

$$\mathbb{E}[Y \mid X = x] = \sum_{y \in R_Y} y\, \mathbb{P}(Y = y \mid X = x), \qquad \mathbb{P}(X = x) > 0.$$

For continuous random variables, the corresponding definition replaces the sum with an integral. The quantity $\mathbb{E}[Y \mid X = x]$ is the average value of Y after restricting to the slice where $X = x$.

Conditional expectation given X depends only on X. For each possible value x, define

$$g(x) := \mathbb{E}[Y \mid X = x].$$

Then

$$\mathbb{E}[Y \mid X] = g(X).$$

For squared-loss regression, the Bayes regressor is

$$x \longmapsto \mathbb{E}[Y \mid X = x].$$

If many data points have features close to x, the best squared-loss prediction is their conditional average response. Many regression methods seek to approximate this unknown function from data.

0.2.3.10 Law of Total Expectation

The *law of total expectation*, also called the tower property, states that

$$\mathbb{E}\Big[\mathbb{E}[Y \mid X]\Big] = \mathbb{E}[Y].$$

The intuition is to condition on X, average Y within each slice $X = x$, and then average over all slices. This identity is used when decomposing expected risk, analyzing algorithms, and proving expected running-time results.

For discrete random variables,

$$\begin{aligned}
\mathbb{E}\Big[\mathbb{E}[Y \mid X]\Big] &= \sum_{x \in R_X} \mathbb{E}[Y \mid X = x]\mathbb{P}(X = x) \\
&= \sum_{x \in R_X} \left(\sum_{y \in R_Y} y\mathbb{P}(Y = y \mid X = x) \right) \mathbb{P}(X = x) \\
&= \sum_{x \in R_X} \sum_{y \in R_Y} y\mathbb{P}(Y = y, X = x) \\
&= \sum_{y \in R_Y} y\mathbb{P}(Y = y) \\
&= \mathbb{E}[Y].
\end{aligned}$$

Example 0.2.7 (Ten coins with different head probabilities)**.** Independently flip ten coins, where coin i has probability $1/i$ of landing heads. Let A be the event that the total number of heads is even, and let B be the event that the number of heads among coins $1, 3, 4, \ldots, 10$ is even. Define the indicator variables

$$Y = \mathbb{1}_A, \qquad X = \mathbb{1}_B.$$

Then $\mathbb{E}[Y] = \mathbb{P}(A)$. Conditional on B, the total number of heads is even exactly when coin 2 lands tails. Conditional on B^c, it is even exactly when coin 2 lands heads. Since coin 2 is fair and independent of the remaining coins,

$$\mathbb{E}[Y \mid X = 1] = \mathbb{P}(A \mid B) = \frac{1}{2}, \qquad \mathbb{E}[Y \mid X = 0] = \mathbb{P}(A \mid B^c) = \frac{1}{2}.$$

Therefore $\mathbb{E}[Y \mid X] = 1/2$, and the law of total expectation gives

$$\mathbb{P}(A) = \mathbb{E}[Y] = \mathbb{E}\Big[\mathbb{E}[Y \mid X]\Big] = \frac{1}{2}.$$

0.2.4 Matrix Analysis

0.2.4.1 Matrix Notation

For real matrices, write

$$\mathcal{M}_{n,d}(\mathbb{R}) = \{n \times d \text{ real matrices}\}, \qquad \mathcal{M}_d(\mathbb{R}) = \mathcal{M}_{d,d}(\mathbb{R}).$$

The set of $d \times d$ orthogonal matrices is

$$\mathcal{O}_d(\mathbb{R}) = \left\{U \in \mathcal{M}_d(\mathbb{R}) : U^T U = U U^T = I_d\right\}.$$

The columns of an orthogonal matrix form an orthonormal basis.

Example 0.2.8 (A data matrix)**.** A dataset with n samples and d features is often stored as

$$X = \begin{bmatrix} - & - & - & x_1^T & - & - \\ - & - & - & x_2^T & - & - \\ & & & \vdots & & \\ - & - & - & x_n^T & - & - \end{bmatrix} \in \mathcal{M}_{n,d}(\mathbb{R}).$$

Matrix analysis helps identify directions of high variance, low-rank structure, conditioning, and stable computation.

0.2.4.2 Eigen-Decomposition of Symmetric Matrices

Let $A \in \mathcal{M}_d(\mathbb{R})$ be symmetric, so $A = A^T$. Then there exist an orthogonal matrix $U \in \mathcal{O}_d(\mathbb{R})$ and a diagonal matrix $D \in \mathcal{M}_d(\mathbb{R})$ such that

$$A = UDU^T.$$

The diagonal entries of D are the eigenvalues of A, and the columns of U are orthonormal eigenvectors of A.

More explicitly, write

$$U = \begin{bmatrix} u_1 & \cdots & u_d \end{bmatrix}, \qquad D = \operatorname{diag}(\lambda_1, \ldots, \lambda_d).$$

Then $A = UDU^T$ means

$$Au_i = \lambda_i u_i, \qquad i = 1, \dots, d,$$

and orthonormality gives

$$u_i^T u_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Geometrically, a symmetric matrix stretches or compresses along mutually orthogonal eigendirections.

Example 0.2.9 (A diagonal symmetric matrix)**.** Let

$$A = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}.$$

Then

$$A = UDU^T, \qquad U = I_2, \qquad D = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}.$$

The eigenvectors are

$$e_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad e_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

with eigenvalues 3 and 1, respectively.

Eigen-decomposition is foundational for:

- principal component analysis, through the variance directions of covariance matrices;
- spectral clustering, through eigenvectors of graph Laplacians;
- optimization, through Hessian eigenvalues and curvature;
- Gaussian models, through covariance structure;
- regularization and conditioning.

The main idea is that a symmetric matrix becomes simple after rotating to its eigenbasis.

0.2.4.3 Positive Semidefinite Matrices

A symmetric matrix $A \in \mathcal{M}_d(\mathbb{R})$ is *positive semidefinite*, written $A \succeq 0$, if

$$x^T A x \geq 0 \qquad \text{for every } x \in \mathbb{R}^d.$$

If

$$A = U \operatorname{diag}(\lambda_1, \dots, \lambda_d) U^T,$$

then the equivalent eigenvalue characterization is

$$A \succeq 0 \quad \iff \quad \lambda_i \geq 0 \text{ for every } i.$$

0.2.4.4 Matrix Square Root

Let $A \succeq 0$ have eigen-decomposition

$$A = U \operatorname{diag}(\lambda_1, \dots, \lambda_d) U^T, \qquad \lambda_i \geq 0.$$

Define its square root by

$$A^{1/2} := U \operatorname{diag}\Big(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_d}\Big) U^T.$$

Then

$$A^{1/2} A^{1/2} = A.$$

Covariance matrices are positive semidefinite, and their square roots appear in Gaussian sampling and whitening transformations.

0.2.4.5 Singular Value Decomposition

Let $A \in \mathcal{M}_{n,d}(\mathbb{R})$. Then there exist orthogonal matrices

$$U \in \mathcal{O}_n(\mathbb{R}), \qquad V \in \mathcal{O}_d(\mathbb{R}),$$

and a rectangular diagonal matrix $D \in \mathcal{M}_{n,d}(\mathbb{R})$ with nonnegative diagonal entries such that

$$A = U D V^T.$$

The diagonal entries of D are the *singular values* of A.

Let $m = \min(n, d)$ and order the singular values as

$$\sigma_1(A) \geq \sigma_2(A) \geq \cdots \geq \sigma_m(A) \geq 0.$$

Writing u_i and v_i for the corresponding left and right singular vectors, respectively, the SVD can also be read as the rank-one expansion

$$A = \sum_{i=1}^{m} \sigma_i(A) u_i v_i^T.$$

Geometrically, a linear map first rotates, then stretches along orthogonal directions by its singular values, and then rotates again. In two dimensions, the image of the unit circle is an ellipse whose principal semiaxes point along the left singular vectors and have lengths given by the singular values.

Reduced SVD Let $r = \operatorname{rank}(A)$. The reduced SVD retains only the nonzero singular values:

$$A = U_r D_r V_r^T,$$

where

$$U_r \in \mathbb{R}^{n \times r}, \qquad D_r = \operatorname{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}, \qquad V_r \in \mathbb{R}^{d \times r}.$$

It stores only the active directions and is therefore more efficient when $r \ll \min(n, d)$.

The SVD is foundational for:

- principal component analysis and dimensionality reduction;
- spectral clustering;
- conditioning and numerical stability;
- matrix completion and recommender systems;
- model compression and low-rank regularization.

Large singular values identify directions in which the matrix acts strongly, while small singular values identify weak or noisy directions.

0.2.4.6 Trace Inner Product and von Neumann's Inequality

For matrices A, B of the same size, define the *Frobenius inner product* by

$$\langle A, B \rangle_F := \operatorname{tr}(A^T B).$$

If $A = (a_{ij})$ and $B = (b_{ij})$, then

$$\operatorname{tr}(A^T B) = \sum_{i,j} a_{ij} b_{ij}.$$

Let $A, B \in \mathcal{M}_{n,d}(\mathbb{R})$, let $m = \min(n, d)$, and order their singular values decreasingly. The *von Neumann trace inequality* states that

$$\operatorname{tr}(A^T B) \leq \sum_{i=1}^{m} \sigma_i(A)\sigma_i(B).$$

Equality holds when the two matrices have aligned left and right singular-vector directions. This is the matrix analogue of Cauchy–Schwarz: the strongest trace alignment is obtained by aligning the strongest singular directions.

0.2.4.7 Connection to Principal Component Analysis

For PCA, a covariance matrix Σ is symmetric and positive semidefinite. The first principal component solves

$$\max_{\|u\|_2=1} u^T \Sigma u.$$

The top p principal directions solve

$$\max_{U^T U = I_p} \operatorname{tr}(U^T \Sigma U).$$

Thus, PCA directions maximize variance subject to orthonormality constraints.

0.2.4.8 Matrix Norms

For $A \in \mathcal{M}_{n,d}(\mathbb{R})$, common matrix norms include the operator norm, Frobenius norm, nuclear norm, and Ky Fan norms. They differ in how they aggregate the singular values of A.

Operator norm The operator norm is

$$\|A\|_{\text{op}} := \max_{\|x\|_2=1} \|Ax\|_2 = \sigma_1(A).$$

It is the largest stretching factor of the linear map $x \mapsto Ax$.

Frobenius norm The Frobenius norm is

$$\|A\|_F := \sqrt{\text{tr}(A^T A)}.$$

Equivalently, with $m = \min(n, d)$,

$$\|A\|_F^2 = \sum_{i,j} a_{ij}^2 = \sum_{i=1}^{m} \sigma_i(A)^2.$$

It is the Euclidean norm of all matrix entries, or equivalently of all singular values.

Nuclear norm The nuclear norm is

$$\|A\|_* := \sum_{i=1}^{m} \sigma_i(A), \qquad m = \min(n, d).$$

It is a convex surrogate for matrix rank, analogous to the use of the ℓ^1 norm to encourage vector sparsity.

0.2.4.9 Low-Rank Approximation

Given $X \in \mathcal{M}_{n,d}(\mathbb{R})$, the best rank-$p$ approximation problem is

$$\min_{\substack{Z \in \mathcal{M}_{n,d}(\mathbb{R}) \\ \text{rank}(Z) \le p}} \|Z - X\|_F .$$

Theorem 0.2.1 (Eckart–Young–Mirsky)**.** *Let*

$$X = U \,\text{diag}(\sigma_1, \ldots, \sigma_m) V^T, \qquad \sigma_1 \ge \cdots \ge \sigma_m \ge 0,$$

be an SVD of X. A best rank-p approximation is obtained by truncating the SVD:

$$X_p = U_{[:,1:p]} \,\text{diag}(\sigma_1, \ldots, \sigma_p) V_{[:,1:p]}^T.$$

Its approximation error is

$$\|X - X_p\|_F^2 = \sum_{i=p+1}^{m} \sigma_i(X)^2.$$

The error is exactly the energy in the discarded singular values. Low-rank approximation is effective when the singular values decay quickly: retaining rank p preserves the first p directions and discards the tail.

Best low-rank approximations are used in PCA and dimensionality reduction, recommender systems and matrix completion, model compression, denoising, and low-rank regularization. Many high-dimensional datasets concentrate their useful structure in a small number of directions.

0.3 Optimization and Unconstrained Optimization

Most supervised-learning problems ask us to choose parameters w that make a loss small. A typical objective is

$$\min_{w\in\mathbb{R}^d} F(w), \qquad F(w) = \frac{1}{n}\sum_{i=1}^{n} \ell\Big(f_w(x_i), y_i\Big) + \lambda R(w).$$

Least squares, logistic regression, support vector machines, matrix factorization, and neural networks can all be formulated as optimization problems.

For example, linear regression uses

$$f_w(x) = \langle w, x\rangle$$

and empirical squared loss

$$F(w) = \frac{1}{2n}\sum_{i=1}^{n}\Big(\langle w, x_i\rangle - y_i\Big)^2.$$

The optimization task is to find

$$w^* \in \arg\min_{w\in\mathbb{R}^d} F(w).$$

0.3.1 Basic Optimization Terminology

0.3.1.1 Infimum and Supremum

Let $I \subseteq \mathbb{R}$. A number $u \in [-\infty, +\infty]$ is a lower bound of I if

$$u \le x \qquad \text{for every } x \in I.$$

The infimum is the greatest lower bound:

$$\inf I := \max\{u \in [-\infty, +\infty] : u \le x \text{ for every } x \in I\}.$$

Similarly, the supremum is the least upper bound:

$$\sup I := \min\{u \in [-\infty, +\infty] : u \ge x \text{ for every } x \in I\}.$$

For example, if $I = (0, 1)$, then

$$\inf I = 0, \qquad \sup I = 1,$$

but neither $\min I$ nor $\max I$ exists.

For an objective function $F\colon \Omega \subseteq \mathbb{R}^d \to \mathbb{R}$, define

$$\inf_{\Omega} F := \inf\{F(x) : x \in \Omega\}, \qquad \sup_{\Omega} F := \sup\{F(x) : x \in \Omega\}.$$

0.3.1.2 Global and Local Optimizers

A point $x^* \in \Omega$ is a *global minimizer* of F on Ω if

$$F(x^*) \leq F(y) \qquad \text{for every } y \in \Omega.$$

A global maximizer satisfies the reversed inequality. We write

$$\arg\min_{\Omega} F := \left\{ x \in \Omega : F(x) = \min_{\Omega} F \right\}.$$

A point $x^* \in \Omega$ is a *local minimizer* if there exists an open ball $B \subseteq \mathbb{R}^d$ containing x^* such that

$$F(x^*) \leq F(y) \qquad \text{for every } y \in B \cap \Omega.$$

A local minimizer only outperforms nearby feasible points; a global minimizer outperforms every feasible point.

0.3.1.3 Optimization Problems and Existence

An optimization problem has the form

$$\min_{x \in \Omega} F(x),$$

where $\Omega \subseteq \mathbb{R}^d$ is the feasible set and $F \colon \Omega \to \mathbb{R}$ is the objective function. In unconstrained optimization, $\Omega = \mathbb{R}^d$ or Ω is open.

An infimum need not be attained. For example, if

$$F(x) = e^x, \qquad x \in \mathbb{R},$$

then $\inf_{\mathbb{R}} F = 0$, but there is no $x \in \mathbb{R}$ for which $e^x = 0$.

Common problem classes include:

Type	**Typical form**
Unconstrained smooth	$F \in C^1$, $\Omega = \mathbb{R}^d$
Smooth constrained	$\Omega = \{x : g_i(x) \leq 0\}$
Convex optimization	Ω convex and F convex
Linear programming	F and g_i affine
Quadratic programming	F quadratic, constraints affine
Combinatorial	Ω finite or discrete

0.3.2 First-Order Necessary Condition

Let $F \colon \Omega \subseteq \mathbb{R}^d \to \mathbb{R}$, where Ω is open. We seek a point

$$x^* \in \arg\min_{\Omega} F.$$

Theorem 0.3.1 (Necessary first-order condition)**.** *Suppose $F \in C^1(\Omega)$ and $x^* \in \Omega$ is a local minimizer of F. Then*

$$\nabla F(x^*) = 0.$$

Proof. Because x^* is a local minimizer and Ω is open, for every unit direction h there is an $\varepsilon_0 > 0$ such that

$$x^* + \varepsilon h \in \Omega \quad \text{and} \quad F(x^* + \varepsilon h) \geq F(x^*)$$

for all sufficiently small $\varepsilon \in [0, \varepsilon_0]$.

Define the one-dimensional function

$$\phi(t) := F(x^* + th).$$

Then

$$\phi'(t) = dF(x^* + th)h,$$

and the fundamental theorem of calculus gives

$$F(x^* + \varepsilon h) - F(x^*) = \int_0^\varepsilon dF(x^* + th)h \, \mathrm{d}t.$$

If $dF(x^*)h < 0$, continuity implies that moving slightly in direction h decreases F, contradicting local minimality. If $dF(x^*)h > 0$, then $dF(x^*)(-h) < 0$, giving the same contradiction in direction $-h$. Hence

$$dF(x^*)h = 0 \qquad \text{for every } h,$$

which is equivalent to $\nabla F(x^*) = 0$. □

0.3.3 Second-Order Optimality Conditions

Taylor's theorem explains the local geometry of optimization. In one dimension,

$$F(x_0 + h) = F(x_0) + F'(x_0)h + \frac{1}{2}F''(x_0)h^2 + o(h^2).$$

The linear term describes the local slope and possible descent directions, while the quadratic term describes curvature. At a smooth local minimum, the first-order term vanishes and the second-order model cannot bend downward.

Theorem 0.3.2 (Necessary second-order condition)**.** *Suppose $F \in C^2(\Omega)$, where $\Omega \subseteq \mathbb{R}^d$ is open, and $x^* \in \Omega$ is a local minimizer. Then*

$$\nabla F(x^*) = 0 \qquad \textit{and} \qquad \nabla^2 F(x^*) \succeq 0.$$

Equivalently,

$$h^T \nabla^2 F(x^*) h \geq 0 \qquad \textit{for every } h \in \mathbb{R}^d.$$

Proof. Fix $h \in \mathbb{R}^d$ and define $\phi(t) = F(x^* + th)$. Since x^* is a local minimizer,

$$\phi(t) - \phi(0) \geq 0$$

for sufficiently small t. Taylor's theorem gives, for some ξ_t between 0 and t,

$$\phi(t) - \phi(0) = t\phi'(0) + \frac{t^2}{2}\phi''(\xi_t).$$

The first-order condition gives $\phi'(0) = 0$. Dividing by $t^2/2$ and letting $t \to 0$ yields

$$\phi''(0) = h^T \nabla^2 F(x^*) h \geq 0.$$

□

Theorem 0.3.3 (Sufficient condition for a local minimum)**.** *Suppose $F \in C^2(\Omega)$. If*

$$\nabla F(x^*) = 0 \qquad \textit{and} \qquad \nabla^2 F(x^*) \succ 0,$$

then x^* is a local minimizer of F.

Positive semidefiniteness of the Hessian is necessary but, by itself, need not be sufficient.

Example 0.3.1 (A saddle point)**.** Consider

$$F(x, y) = x^2 - y^2.$$

At the origin,

$$\nabla F(0,0) = 0, \qquad \nabla^2 F(0,0) = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}.$$

The Hessian is indefinite, so $(0,0)$ is a saddle point rather than a local minimum.

These conditions describe local structure only. Objectives arising in neural-network training, matrix factorization, and deep reinforcement learning can be nonconvex, so a local minimizer need not be global.

0.3.4 Convexity and Global Optimality

A function

$$F \colon \mathbb{R}^d \to (-\infty, +\infty]$$

is *convex* if

$$F\Big((1-\lambda)x + \lambda y\Big) \leq (1-\lambda)F(x) + \lambda F(y)$$

for all $x, y \in \mathbb{R}^d$ and $\lambda \in [0,1]$. If the inequality is strict whenever $x \neq y$ and $\lambda \in (0,1)$, then F is *strictly convex*. Geometrically, the graph of a convex function lies below every chord joining two points on its graph.

Least squares and logistic regression have convex objectives, whereas neural-network training is typically nonconvex.

Theorem 0.3.4 (Local minima of convex functions are global)**.** *If F is convex, then every local minimizer of F is a global minimizer.*

Proof. Suppose x^* is a local minimizer but is not global. Then there is some y such that $F(y) < F(x^*)$. For $\lambda \in (0,1)$, convexity gives

$$F\Big((1-\lambda)x^* + \lambda y\Big) \le (1-\lambda)F(x^*) + \lambda F(y) < F(x^*).$$

For sufficiently small λ, the point $(1-\lambda)x^* + \lambda y$ is arbitrarily close to x^*, contradicting local minimality. □

0.3.5 Gradient-Based Methods

Suppose we can evaluate $F(x_t)$ and $\nabla F(x_t)$ at one point at a time. An iterative method has the form

$$x_{t+1} = x_t + \alpha_t h_t,$$

where h_t is a direction and $\alpha_t > 0$ is a step size. The gradient points uphill, while the negative gradient points downhill; the step size controls how far the algorithm moves.

0.3.5.1 Directions of Descent

A nonzero vector $h \in \mathbb{R}^d$ is a *direction of descent* for F at x if there exists $\varepsilon_0 > 0$ such that

$$F(x + \varepsilon h) < F(x) \qquad \text{for every } \varepsilon \in (0, \varepsilon_0].$$

If $F \in C^1$ and

$$h^T \nabla F(x) < 0,$$

then h is a direction of descent. Indeed, the first-order expansion gives

$$F(x + \varepsilon h) - F(x) = \varepsilon h^T \nabla F(x) + o(\varepsilon).$$

The leading term is negative, so the objective decreases for sufficiently small positive ε.

The negative-gradient direction

$$h = -\nabla F(x)$$

satisfies

$$h^T \nabla F(x) = -\left\|\nabla F(x)\right\|_2^2 < 0$$

whenever $\nabla F(x) \neq 0$. It is the steepest descent direction in the Euclidean norm.

0.3.5.2 Quadratic Geometry

Euclidean gradient descent can zig-zag on ill-conditioned objectives. Preconditioning and Newton-type methods attempt to correct this geometry.

For example, let

$$F(x) = \frac{1}{2} x^T A x - b^T x, \qquad A \in S_d^{++}.$$

Then

$$\nabla F(x) = Ax - b,$$

and the unique minimizer satisfies

$$Ax^* = b, \qquad x^* = A^{-1}b.$$

Plain gradient descent uses

$$x_{t+1} = x_t - \alpha_t(Ax_t - b).$$

0.3.5.3 Step Size and Line Search

Even when h_t is a descent direction, an excessively large step can increase the objective. A line search chooses

$$\alpha \in (0, \alpha_{\max}]$$

so that $F(x + \alpha h)$ decreases sufficiently. Exact line search minimizes $F(x + \alpha h)$ over α, while approximate line searches use conditions such as the Armijo or Wolfe rules.

0.3.6 Stochastic Gradient Methods

In machine learning, an empirical objective often has the form

$$F(w) = \frac{1}{n}\sum_{i=1}^{n} F_i(w), \qquad \nabla F(w) = \frac{1}{n}\sum_{i=1}^{n} \nabla F_i(w).$$

When n is large, computing the full sum at every iteration is expensive, so a randomly selected subset of the data is used to approximate the gradient.

For least squares,

$$F(w) = \frac{1}{2n}\sum_{i=1}^{n}\Big(\langle w, x_i\rangle - y_i\Big)^2,$$

and

$$\nabla F(w) = \frac{1}{n}\sum_{i=1}^{n}\Big(\langle w, x_i\rangle - y_i\Big)x_i.$$

Choose a random mini-batch $I_t \subseteq \{1, \ldots, n\}$ with $|I_t| \ll n$, and define

$$g_t = \frac{1}{|I_t|}\sum_{i\in I_t}\Big(\langle w_t, x_i\rangle - y_i\Big)x_i.$$

The mini-batch update is

$$w_{t+1} = w_t - \alpha_t g_t.$$

A typical step-size condition for convergence is

$$\sum_{t=1}^{\infty}\alpha_t = \infty, \qquad \sum_{t=1}^{\infty}\alpha_t^2 < \infty.$$

The first condition ensures that the iterates keep moving far enough to reach the target, while the second allows stochastic noise to be averaged out. A common example is $\alpha_t = 1/t$.

Optimization appears across machine learning in several forms:

Method	**Objective**	**Optimization type**
Least squares	squared error	convex, smooth
Ridge regression	squared error $+\ell^2$ penalty	strongly convex if $\lambda > 0$
Logistic regression	cross-entropy	convex, smooth
SVM	hinge loss + regularization	convex, nonsmooth
Lasso	squared error $+\ell^1$ penalty	convex, nonsmooth
Neural networks	empirical loss	typically nonconvex

Optimization is therefore not merely solving equations; it is the design of dynamics that move toward useful regions of a loss landscape.

0.3.6.1 SGD Example: Computing a Mean

Given numbers $y_1, \dots, y_n$, consider

$$F(w) = \frac{1}{n}\sum_{i=1}^{n} F_i(w), \qquad F_i(w) = \frac{1}{2}(w - y_i)^2.$$

Since

$$\nabla F(w) = \frac{1}{n}\sum_{i=1}^{n}(w - y_i) = w - \bar{y}, \qquad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i,$$

the minimizer is $w^* = \bar{y}$.

At iteration t, draw i_t uniformly from $\{1, \dots, n\}$. Because $\nabla F_{i_t}(w) = w - y_{i_t}$, the SGD update is

$$w_{t+1} = w_t - \alpha_t(w_t - y_{i_t}) = (1 - \alpha_t)w_t + \alpha_t y_{i_t}.$$

Thus each iteration moves w_t a fraction α_t of the way toward a randomly selected observation.

With $\alpha_t = 1/t$, the iterate becomes the running average

$$w_{t+1} = \frac{y_{i_1} + \cdots + y_{i_t}}{t}.$$

The sampled values are i.i.d. with mean $\bar{y}$, so the strong law of large numbers gives

$$w_t \longrightarrow \bar{y} = w^* \qquad \text{almost surely.}$$

Other step-size choices illustrate why both summability conditions matter. A constant step size violates $\sum_t \alpha_t^2 < \infty$, so stochastic fluctuations do not die out. The rapidly decaying choice $\alpha_t = 1/t^2$ violates $\sum_t \alpha_t = \infty$, so the iterates may stop moving before reaching the target.

0.4 Introduction to Learning and Kernel Density Estimation

We observe finitely many data points from an unknown process and use them to learn about unseen data. Two basic tasks are:

- *supervised learning*: observe input–output pairs (X_i, Y_i) and learn to predict Y from X;
- *generative modeling*: observe inputs X_i and learn their distribution.

In supervised learning, the training set is

$$\mathcal{T} = \{(X_1, Y_1), \dots, (X_n, Y_n)\},$$

where the pairs are i.i.d. from an unknown distribution. The goal is to predict the response for a new pair (X, Y) drawn from the same distribution. Examples include medical-risk prediction, spam filtering, house-price prediction, and activity recognition.

For generative modeling, suppose

$$\mathcal{T} = \{X_1, \dots, X_n\}, \qquad X_1, \dots, X_n \overset{\text{iid}}{\sim} f,$$

where f is an unknown density. A simple pipeline first estimates f from the observations and then samples new data from the estimated density. Applications include generation of images, text, audio, molecules, weather fields, and trajectories.

0.4.1 Kernels and Bandwidth

Definition 0.4.1 (Kernel)**.** A function $K\colon \mathbb{R}^d \to [0, \infty)$ is a *kernel* if:

1. it integrates to one,
$$\int_{\mathbb{R}^d} K(x)\, \mathrm{d}x = 1;$$
2. it is symmetric, $K(x) = K(-x)$;
3. it satisfies a mild integrability condition, such as
$$\int_{\mathbb{R}^d} \|x\|_2 \, K(x)\, \mathrm{d}x < \infty.$$

A common choice is the Gaussian kernel

$$K(x) = \frac{1}{(2\pi)^{d/2}} \exp\left(-\frac{\|x\|_2^2}{2}\right).$$

In one dimension this reduces to $K(x) = (2\pi)^{-1/2} e^{-x^2/2}$.

Given a bandwidth $\delta > 0$, define the rescaled kernel

$$K_\delta(x) := \frac{1}{\delta^d} K\left(\frac{x}{\delta}\right).$$

A small bandwidth produces a narrow, local bump, while a large bandwidth produces a wider, smoother bump.

Proposition 0.4.1. *If K is a kernel, then K_δ is also a kernel.*

Proof. Nonnegativity is immediate. With the substitution $y = x/\delta$, so that $\mathrm{d}x = \delta^d \,\mathrm{d}y$,

$$\int_{\mathbb{R}^d} K_\delta(x)\ \mathrm{d}x = \int_{\mathbb{R}^d} \frac{1}{\delta^d} K\left(\frac{x}{\delta}\right)\ \mathrm{d}x = \int_{\mathbb{R}^d} K(y)\ \mathrm{d}y = 1.$$

Symmetry follows from

$$K_\delta(-x) = \frac{1}{\delta^d} K\left(-\frac{x}{\delta}\right) = K_\delta(x).$$

Finally,

$$\int_{\mathbb{R}^d} \|x\|_2 \, K_\delta(x)\ \mathrm{d}x = \delta \int_{\mathbb{R}^d} \|y\|_2 \, K(y)\ \mathrm{d}y < \infty.$$

□

0.4.2 Kernel Density Estimation

Given $X_1, \ldots, X_n$, a kernel K, and a bandwidth $\delta > 0$, the *kernel density estimator* is

$$\widehat{f}_\delta(x) := \frac{1}{n} \sum_{i=1}^n K_\delta(x - X_i) = \frac{1}{n\delta^d} \sum_{i=1}^n K\left(\frac{x - X_i}{\delta}\right).$$

At location x, every observation contributes a bump centered at X_i, and the estimate averages these bumps. KDE therefore converts discrete sample points into a smooth density estimate.

Bandwidth controls the statistical tradeoff:

Bandwidth	**Appearance**	**Statistical behavior**
Small δ	very wiggly	low bias, high variance
Medium δ	balanced	often useful
Large δ	very smooth	high bias, low variance

0.4.2.1 Basic Properties

The KDE is nonnegative because every kernel term is nonnegative:

$$\widehat{f}_\delta(x) \geq 0.$$

It also integrates to one:

$$\begin{aligned}\int_{\mathbb{R}^d} \widehat{f}_\delta(x)\ \mathrm{d}x &= \frac{1}{n} \sum_{i=1}^n \int_{\mathbb{R}^d} K_\delta(x - X_i)\ \mathrm{d}x \\ &= \frac{1}{n} \sum_{i=1}^n 1 = 1.\end{aligned}$$

Thus $\widehat{f}_\delta$ is itself a density.

The KDE also preserves the sample mean:

$$\int_{\mathbb{R}^d} x\widehat{f}_\delta(x)\ \mathrm{d}x = \frac{1}{n}\sum_{i=1}^{n} X_i = \overline{X}_n.$$

Indeed, with $y = x - X_i$,

$$\begin{aligned}\int_{\mathbb{R}^d} xK_\delta(x - X_i)\ \mathrm{d}x &= \int_{\mathbb{R}^d} (y + X_i)K_\delta(y)\ \mathrm{d}y \\ &= X_i,\end{aligned}$$

because symmetry gives $\int yK_\delta(y)\ \mathrm{d}y = 0$ and $\int K_\delta(y)\ \mathrm{d}y = 1$. Hence the center of mass of the KDE is the empirical center of the data.

0.4.3 Pointwise Error and Bias–Variance

To emphasize its dependence on the random training sample, write

$$\widehat{f}_\delta(x; \mathcal{T}), \qquad \mathcal{T} = (X_1, \ldots, X_n).$$

For fixed x and δ, this is a random variable. Its pointwise mean squared error is

$$\mathrm{MSE}(x) := \mathbb{E}_{\mathcal{T}}\left[\left(\widehat{f}_\delta(x; \mathcal{T}) - f(x)\right)^2\right].$$

This criterion evaluates the density estimate at one fixed location x, averaging only over the random sample.

The bias–variance decomposition is

$$\mathrm{MSE}(x) = \mathrm{Var}\Big(\widehat{f}_\delta(x; \mathcal{T})\Big) + \Big(\mathbb{E}\Big[\widehat{f}_\delta(x; \mathcal{T})\Big] - f(x)\Big)^2.$$

Variance measures randomness caused by the sample, whereas bias measures the systematic smoothing error caused by the bandwidth. Small bandwidth typically produces low bias and high variance; large bandwidth typically produces high bias and low variance.

Proof of the bias–variance decomposition. Fix x and write

$$Z_x := \widehat{f}_\delta(x; \mathcal{T}), \qquad B := \mathbb{E}[Z_x] - f(x).$$

Adding and subtracting $\mathbb{E}[Z_x]$ gives

$$\begin{aligned}\mathbb{E}\Big[(Z_x - f(x))^2\Big] &= \mathbb{E}\Big[(Z_x - \mathbb{E}[Z_x] + B)^2\Big] \\ &= \mathbb{E}\Big[(Z_x - \mathbb{E}[Z_x])^2\Big] + 2B\mathbb{E}[Z_x - \mathbb{E}[Z_x]] + B^2 \\ &= \mathrm{Var}(Z_x) + B^2.\end{aligned}$$

□

0.4.4 Bias and Variance Bounds for KDE

Theorem 0.4.1 (KDE bias)**.** *Assume f is twice differentiable and*

$$\left\|\nabla^2 f(y)\right\|_{\mathrm{op}} \le L \qquad \text{for every } y,$$

and suppose

$$\mu_2 := \int_{\mathbb{R}^d} \|z\|_2^2 K(z)\, \mathrm{d}z < \infty.$$

Then, for every $x \in \mathbb{R}^d$ and $\delta > 0$,

$$\left|\mathbb{E}[\widehat{f}_\delta(x)] - f(x)\right| \le \frac{L\mu_2}{2}\delta^2.$$

Consequently, the squared bias is $O(\delta^4)$.

Proof. Since the observations have common density f,

$$\begin{aligned}\mathbb{E}[\widehat{f}_\delta(x)] &= \int_{\mathbb{R}^d} K_\delta(x-y) f(y)\, \mathrm{d}y \\ &= \int_{\mathbb{R}^d} K(z) f(x-\delta z)\, \mathrm{d}z.\end{aligned}$$

Therefore,

$$\mathbb{E}[\widehat{f}_\delta(x)] - f(x) = \int_{\mathbb{R}^d} K(z)\Big(f(x-\delta z) - f(x)\Big)\, \mathrm{d}z.$$

A second-order Taylor expansion gives

$$f(x-\delta z) - f(x) = -\delta \nabla f(x)^T z + \frac{\delta^2}{2} z^T \nabla^2 f(\xi_z) z$$

for an intermediate point ξ_z. Symmetry of K implies $\int zK(z)\, \mathrm{d}z = 0$, so the first-order term vanishes. Using the Hessian bound,

$$\left|\mathbb{E}[\widehat{f}_\delta(x)] - f(x)\right| \le \frac{L\delta^2}{2} \int_{\mathbb{R}^d} \|z\|_2^2 K(z)\, \mathrm{d}z.$$

□

More generally, if f has bounded derivatives through order $s \ge 2$ and K is a matching higher-order kernel, then

$$\left|\mathbb{E}[\widehat{f}_\delta(x)] - f(x)\right| = O(\delta^s), \qquad \text{squared bias} = O(\delta^{2s}).$$

Theorem 0.4.2 (KDE variance)**.** *Assume $X_1, \ldots, X_n$ are i.i.d. with density f, where $f(y) \le M$ for every y, and assume*

$$\int_{\mathbb{R}^d} K(z)^2\, \mathrm{d}z < \infty.$$

Then, for every $x \in \mathbb{R}^d$, $\delta > 0$, and $n \ge 1$,

$$\mathrm{Var}\Big(\widehat{f}_\delta(x)\Big) \le \frac{M}{n\delta^d} \int_{\mathbb{R}^d} K(z)^2\, \mathrm{d}z.$$

Proof. Independence and identical distribution give

$$\mathrm{Var}\Big(\widehat{f}_\delta(x)\Big) = \frac{1}{n}\mathrm{Var}\Big(K_\delta(x-X)\Big), \qquad X \sim f.$$

Dropping the squared mean and substituting $z = (x-y)/\delta$,

$$\begin{aligned}\mathrm{Var}\Big(K_\delta(x-X)\Big) &\le \mathbb{E}\Big[K_\delta(x-X)^2\Big] \\ &= \frac{1}{\delta^d}\int_{\mathbb{R}^d} K(z)^2 f(x-\delta z)\,\mathrm{d}z \\ &\le \frac{M}{\delta^d}\int_{\mathbb{R}^d} K(z)^2\,\mathrm{d}z.\end{aligned}$$

□

Thus larger n and larger δ reduce variance, whereas higher dimension worsens it.

0.4.5 Bandwidth Rate and the Curse of Dimensionality

Under the preceding smoothness assumptions, the pointwise MSE has the form

$$\mathrm{MSE}(x) \le \frac{C_1}{n\delta^d} + C_2\delta^{2s}.$$

Balancing variance and squared bias gives

$$\frac{1}{n\delta^d} \asymp \delta^{2s}, \qquad \delta \asymp n^{-1/(2s+d)}.$$

At this bandwidth, both terms have the same order:

$$\mathrm{MSE}(x) \asymp n^{-2s/(2s+d)}.$$

Greater smoothness s improves the rate, while greater dimension d worsens it.

To obtain MSE of order at most ε, the required sample size scales as

$$n \asymp \varepsilon^{-(2s+d)/(2s)}.$$

The exponent grows with d, which is the *curse of dimensionality*. As dimension increases, space expands quickly, observations become sparse, and local averaging becomes unstable.

A common working assumption is the *manifold hypothesis*: high-dimensional real data concentrate near a much lower-dimensional set and therefore have only a few effective degrees of freedom. This is a hypothesis rather than a theorem. When it holds, learning methods can exploit intrinsic rather than ambient dimension, motivating manifold learning, representation learning, and modern generative models.

0.5 Prediction Basics and the Bayes Predictor

Let $X \in \mathcal{R}_X$ be an input and $Y \in \mathcal{R}_Y$ an output. We observe

$$\mathcal{T} = \Big((X_1, Y_1), \ldots, (X_n, Y_n)\Big),$$

where the pairs are i.i.d. from an unknown distribution ρ. The goal is to construct a predictor

$$f \colon \mathcal{R}_X \to \mathcal{R}_Y$$

that predicts Y from X on a new pair drawn from ρ.

In regression, $\mathcal{R}_Y \subseteq \mathbb{R}$ and the response is numerical. In classification, $\mathcal{R}_Y$ is finite and the response is a category. Both use the same prediction framework; the main difference is the output space and the loss function.

0.5.1 Loss Functions

A loss function

$$r \colon \mathcal{R}_Y \times \mathcal{R}_Y \to [0, \infty)$$

measures the quality of a prediction. If Y is the true output and $f(X)$ is the prediction, the incurred loss is $r(Y, f(X))$. The choice of loss determines what constitutes the best predictor.

For regression, a common choice is squared loss:

$$r(y, \widehat{y}) = (y - \widehat{y})^2.$$

For classification, the zero–one loss is

$$r(y, \widehat{y}) = \begin{cases} 0, & y = \widehat{y}, \\ 1, & y \neq \widehat{y}. \end{cases}$$

If different errors have different consequences, a cost-sensitive loss is more appropriate. For example, with labels $\{\text{healthy}, \text{sick}\}$, one may assign loss 1 to a false positive and loss 10 to a false negative.

0.5.1.1 Probabilistic Classification

For categorical Y, a predictor may output a probability distribution rather than a hard label:

$$f \colon \mathcal{R}_X \to \Delta(\mathcal{R}_Y), \qquad f(x, y) \geq 0, \qquad \sum_{y \in \mathcal{R}_Y} f(x, y) = 1.$$

The output $f(x, y)$ approximates $\mathbb{P}(Y = y \mid X = x)$. A common loss is the negative log-likelihood

$$r\Big(y, f(x, \cdot)\Big) = -\log f(x, y),$$

which depends on the probability assigned to the true class. A confident incorrect prediction receives a large loss, while a confident correct prediction receives loss near zero.

0.5.2 Expected Risk

The *expected risk*, also called the generalization error, of a predictor f is

$$R(f) := \mathbb{E}_{(X,Y)\sim\rho}\Big[r(Y, f(X))\Big].$$

It is the average loss on a new pair from the population distribution. A good predictor has small expected risk.

A *Bayes predictor* is a minimizer over all predictors:

$$f^* \in \arg\min_f R(f) = \arg\min_f \mathbb{E}\Big[r(Y, f(X))\Big].$$

It is the best possible predictor for the chosen loss.

0.5.3 Conditional Risk and the Pointwise Rule

For a fixed input x and prediction $y' \in \mathcal{R}_Y$, define the *conditional risk*

$$\mathbb{E}\Big[r(Y, y') \mid X = x\Big].$$

Only Y is random: this is the average loss of predicting y' over labels that may occur with input x. If $\mathcal{R}_Y$ is finite, then

$$\mathbb{E}\Big[r(Y, y') \mid X = x\Big] = \sum_{y\in\mathcal{R}_Y} r(y, y')\mathbb{P}(Y = y \mid X = x).$$

Theorem 0.5.1 (Pointwise characterization of the Bayes predictor)**.** *Suppose f^* satisfies*

$$f^*(x) \in \arg\min_{y'\in\mathcal{R}_Y} \mathbb{E}\Big[r(Y, y') \mid X = x\Big] \qquad \text{for every } x.$$

Then f^* is a Bayes predictor: for every predictor g,

$$R(f^*) \le R(g).$$

Proof. The law of total expectation gives

$$R(f) = \mathbb{E}\left[\mathbb{E}\Big[r(Y, f(X)) \mid X\Big]\right].$$

For each fixed x, the inner conditional expectation depends on f only through the single value $f(x)$. By the definition of $f^*(x)$,

$$\mathbb{E}\Big[r(Y, f^*(x)) \mid X = x\Big] \le \mathbb{E}\Big[r(Y, g(x)) \mid X = x\Big].$$

Substituting $x = X$ and taking expectations yields $R(f^*) \le R(g)$. □

0.5.4 Bayes Regression under Squared Loss

Assume $\mathcal{R}_Y \subseteq \mathbb{R}$, use squared loss, and suppose $\mathbb{E}[Y^2] < \infty$. Then the Bayes predictor is the conditional mean:

$$f^*(x) = \mathbb{E}[Y \mid X = x].$$

Thus, among all observations having features x, the ideal prediction is their average response.

Theorem 0.5.2 (Squared-loss Bayes predictor)**.** *For the loss $r(y, y') = (y - y')^2$,*

$$f^*(x) = \mathbb{E}[Y \mid X = x]$$

minimizes the conditional risk at every x.

Proof. Fix x and write $m(x) = \mathbb{E}[Y \mid X = x]$. For any prediction $y' \in \mathbb{R}$, add and subtract the conditional mean:

$$Y - y' = \Big(Y - m(x)\Big) + \Big(m(x) - y'\Big).$$

Expanding the square and conditioning on $X = x$ gives

$$\mathbb{E}\Big[(Y - y')^2 \mid X = x\Big] = \mathbb{E}\Big[(Y - m(x))^2 \mid X = x\Big] + 2\Big(m(x) - y'\Big)\mathbb{E}\Big[Y - m(x) \mid X = x\Big] + \Big(m(x) - y'\Big)^2.$$

The middle term vanishes because

$$\mathbb{E}[Y - m(x) \mid X = x] = 0.$$

Hence

$$\mathbb{E}\Big[(Y - y')^2 \mid X = x\Big] = \mathbb{E}\Big[(Y - m(x))^2 \mid X = x\Big] + \Big(m(x) - y'\Big)^2.$$

The first term does not depend on y', and the second is minimized at $y' = m(x)$. The pointwise characterization therefore proves the result. □

The identity in the proof is also a conditional bias–variance decomposition: departing from the conditional mean adds the nonnegative penalty $\Big(m(x) - y'\Big)^2$.

0.5.5 Bayes Classification under Zero–One Loss

Assume $\mathcal{R}_Y$ is finite and

$$r(y, y') = \mathbf{1}\{y \neq y'\}.$$

Then a Bayes classifier predicts a most probable label conditional on the observed features:

$$f^*(x) \in \arg\max_{y \in \mathcal{R}_Y} \mathbb{P}(Y = y \mid X = x).$$

This is the *maximum a posteriori*, or MAP, rule. The prior $\mathbb{P}(Y = y)$ ignores the observed input, whereas the posterior $\mathbb{P}(Y = y \mid X = x)$ incorporates it.

Proof. For fixed x and a proposed label y',

$$\begin{aligned}\mathbb{E}\Big[\mathbf{1}\{Y \neq y'\} \mid X = x\Big] &= \mathbb{P}(Y \neq y' \mid X = x) \\ &= 1 - \mathbb{P}(Y = y' \mid X = x).\end{aligned}$$

Minimizing this expression is equivalent to maximizing the posterior probability of the predicted label. □

For example, suppose the posterior probabilities of the labels low, medium, and high risk at a patient's feature vector x are respectively 0.20, 0.35, and 0.45. Under zero–one loss, the Bayes classifier predicts high risk. A probabilistic classifier can instead report the entire vector of posterior probabilities, which preserves uncertainty that a hard MAP decision hides.

0.5.6 From the Bayes Rule to a Learning Algorithm

Both ideal rules depend on the unknown conditional distribution of Y given $X = x$:

$$\mathbb{E}[Y \mid X = x] \qquad \text{or} \qquad \mathbb{P}(Y = y \mid X = x).$$

The Bayes rule is therefore an ideal target, not usually a directly available algorithm. Learning is difficult because the joint distribution of (X, Y) must be inferred from finitely many observations.

Writing the training data as

$$\mathcal{T} = \Big((X_1, Y_1), \ldots, (X_n, Y_n)\Big),$$

a learning algorithm is a map

$$\mathcal{A} \colon \mathcal{T} \longmapsto f_{\mathcal{T}}.$$

Two broad construction strategies are:

1. *model-based prediction*: estimate the joint or conditional distribution and plug the estimate into the Bayes rule;
2. *empirical risk minimization*: choose a predictor from a prescribed function class by minimizing its average training loss.

Gaussian models, naive Bayes, kernel density estimation, and kernel regression are examples of the first strategy. The second strategy is developed next.

0.5.7 Joint Gaussian Model for Regression

Let $X \in \mathbb{R}^d$ and $Y \in \mathbb{R}$, and suppose the joint vector is Gaussian:

$$\begin{pmatrix} X \\ Y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}, \begin{pmatrix} \Sigma_{XX} & \Sigma_{XY} \\ \Sigma_{YX} & \Sigma_{YY} \end{pmatrix}\right).$$

Here Σ_{XX} describes variation among the features, while Σ_{YX} records how the response co-varies with each feature.

Theorem 0.5.3 (Gaussian conditional mean)**.** *If Σ_{XX} is invertible, then the conditional distribution of Y given $X = x$ is Gaussian and*

$$\mathbb{E}[Y \mid X = x] = \mu_Y + \Sigma_{YX}\Sigma_{XX}^{-1}(x - \mu_X).$$

Consequently, the squared-loss Bayes predictor is affine in x.

Writing

$$\beta = \Sigma_{YX}\Sigma_{XX}^{-1}, \qquad b = \mu_Y - \beta\mu_X,$$

the predictor becomes

$$f^*(x) = b + \beta x.$$

Thus linear regression can be interpreted as the Bayes predictor under a joint Gaussian model. In one dimension, the slope is

$$\beta = \frac{\mathrm{Cov}(X, Y)}{\mathrm{Var}(X)}.$$

The theorem follows by writing the joint Gaussian density, fixing x, and completing the square in y. Terms depending only on x enter the normalizing constant; the remaining quadratic in y is the kernel of a Gaussian density.

In practice, replace the unknown parameters with their sample estimates:

$$\widehat{\mu}_X = \frac{1}{n}\sum_{i=1}^{n} X_i, \qquad \widehat{\mu}_Y = \frac{1}{n}\sum_{i=1}^{n} Y_i,$$

and similarly estimate Σ_{XX} and Σ_{YX}. The plug-in predictor is

$$\widehat{f}(x) = \widehat{\mu}_Y + \widehat{\Sigma}_{YX}\widehat{\Sigma}_{XX}^{-1}(x - \widehat{\mu}_X).$$

0.5.8 Naive Bayes

For classification, a simpler distributional model assumes that the feature coordinates $X = (X^{(1)}, \ldots, X^{(d)})$ are conditionally independent given the class. Then

$$p(x \mid Y = y) = \prod_{j=1}^{d} p(x^{(j)} \mid Y = y).$$

Bayes' rule gives

$$\mathbb{P}(Y = y \mid X = x) = \frac{p(x \mid Y = y)\mathbb{P}(Y = y)}{p(x)}.$$

Because $p(x)$ is the same for every candidate class,

$$\mathbb{P}(Y = y \mid X = x) \propto \mathbb{P}(Y = y) \prod_{j=1}^{d} p(x^{(j)} \mid Y = y).$$

The MAP classifier therefore selects the class maximizing the right-hand side.

Conditional independence is often unrealistic, but it reduces estimation of a high-dimensional conditional density to estimation of one-dimensional distributions. This simplicity can be valuable with limited data; text and spam classification are standard examples.

0.5.9 Kernel Regression from a Joint Density Estimate

Another model-based approach estimates the joint density of (X, Y) by KDE and plugs it into the conditional-mean Bayes rule. For $X \in \mathbb{R}^d$ and $Y \in \mathbb{R}$, let

$$\widehat{p}_{X,Y}(x, y) = \frac{1}{n} \sum_{i=1}^{n} K_h\Big((x, y) - (X_i, Y_i)\Big).$$

Suppose the joint kernel factors as

$$K_h\Big((x, y) - (X_i, Y_i)\Big) = K_{X,h}(x - X_i) K_{Y,h}(y - Y_i).$$

Integrating out y gives the marginal estimate

$$\widehat{p}_X(x) = \frac{1}{n} \sum_{j=1}^{n} K_{X,h}(x - X_j),$$

so the estimated conditional density is the mixture

$$\widehat{p}(y \mid x) = \sum_{i=1}^{n} w_i(x) K_{Y,h}(y - Y_i), \qquad w_i(x) = \frac{K_{X,h}(x - X_i)}{\sum_{j=1}^{n} K_{X,h}(x - X_j)}.$$

The weights are nonnegative and sum to one; training inputs nearer to x get larger weight.

If the Y-kernel is centered, then

$$\int_{\mathbb{R}} y K_{Y,h}(y - Y_i) \, \mathrm{d}y = Y_i.$$

Therefore the plug-in estimate of the squared-loss Bayes predictor is

$$\begin{aligned} \widehat{f}(x) &= \int_{\mathbb{R}} y \widehat{p}(y \mid x) \, \mathrm{d}y \\ &= \sum_{i=1}^{n} w_i(x) Y_i \\ &= \frac{\sum_{i=1}^{n} K_{X,h}(x - X_i) Y_i}{\sum_{j=1}^{n} K_{X,h}(x - X_j)}. \end{aligned}$$

This is the *Nadaraya–Watson kernel regression estimator*: a local weighted average of responses whose weights depend only on proximity in the feature space.

As in density estimation, the bandwidth controls a bias–variance tradeoff. A small bandwidth produces highly local predictions with low bias but high variance. A large bandwidth averages more observations, reducing variance at the cost of greater bias.

0.5.10 The Empirical Distribution and Memorization

A particularly rough estimate of the population distribution is the empirical distribution

$$\widehat{P}_{X,Y} = \frac{1}{n} \sum_{i=1}^{n} \delta_{(X_i, Y_i)},$$

where δ_z is a point mass at z. Plugging this estimate directly into a pointwise prediction rule can simply memorize the data:

$$\widehat{f}(X_i) = Y_i \qquad \text{for every training observation } i.$$

It fits the observed inputs exactly but gives no principled prescription for a new input. This motivates either smoothing the distribution estimate or restricting the set of admissible predictors.

0.5.11 Empirical Risk Minimization

The population risk

$$R(f) = \mathbb{E}[r(Y, f(X))]$$

is generally unavailable because the distribution of (X, Y) is unknown. Its observable analogue is the *empirical risk*

$$\widehat{R}_{\mathcal{T}}(f) = \frac{1}{n}\sum_{i=1}^{n} r(Y_i, f(X_i)).$$

Given a function class $\mathcal{F}$, an *empirical risk minimizer* is

$$\widehat{f}_{\mathcal{T}} \in \arg\min_{f\in\mathcal{F}} \widehat{R}_{\mathcal{T}}(f).$$

ERM replaces the unknown population average by the corresponding average over the training sample.

0.5.11.1 Least-Squares Regression as ERM

For the affine function class

$$\mathcal{F} = \{x \mapsto w^\top x + b : w \in \mathbb{R}^d,\ b \in \mathbb{R}\}$$

and squared loss, ERM becomes

$$(\widehat{w}, \widehat{b}) \in \arg\min_{w,b} \frac{1}{n}\sum_{i=1}^{n}\Big(Y_i - (w^\top X_i + b)\Big)^2.$$

Thus ordinary least squares is squared-loss ERM over affine predictors.

0.5.11.2 Classification as ERM

With zero–one loss, ERM solves

$$\widehat{f}_{\mathcal{T}} \in \arg\min_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{Y_i \neq f(X_i)\}.$$

The class $\mathcal{F}$ might consist of linear classifiers, decision trees, kernel classifiers, or neural networks. The same ERM principle applies, but the choice of function class changes both computation and generalization.

0.5.12 Why the Function Class Matters

If $\mathcal{F}$ contains every function from $\mathcal{R}_X$ to $\mathcal{R}_Y$, ERM can memorize the sample. For example, suppose the inputs $X_1, \ldots, X_n$ are distinct and $Y \in \mathbb{R}$. Define

$$\widehat{f}_{\mathcal{T}}(x) = \begin{cases} Y_i, & x = X_i \text{ for some } i, \\ 0, & \text{otherwise.} \end{cases}$$

Its squared training error is zero, yet its value at any unseen input is unrelated to the underlying regression function. Zero training error is therefore not the same as good prediction.

Restricting $\mathcal{F}$ controls flexibility:

- if $\mathcal{F}$ is too small, it may not approximate the true relationship well;
- if $\mathcal{F}$ is too large, it may fit random features of the training sample and become unstable.

This is the model-flexibility version of the bias–variance tradeoff. Increasing flexibility generally reduces approximation bias but can increase variance.

0.5.13 Regularized ERM

Rather than restricting the function class sharply, one can penalize complex predictors:

$$\widehat{f}_{\mathcal{T}} \in \arg\min_{f \in \mathcal{F}} \left\{ \frac{1}{n} \sum_{i=1}^{n} r(Y_i, f(X_i)) + \lambda \Omega(f) \right\}.$$

The first term rewards fit to the observed data, while $\Omega(f)$ discourages complexity and $\lambda \geq 0$ controls the tradeoff. For linear predictors, ridge uses $\Omega(w) = \|w\|_2^2$ and lasso uses $\Omega(w) = \|w\|_1$. Weight decay in neural networks serves the same broad purpose.

0.5.14 Training Error Is Not Test Error

For a fixed predictor f, define

$$Z_i = r(Y_i, f(X_i)).$$

The variables Z_i are i.i.d., and the law of large numbers gives

$$\widehat{R}_{\mathcal{T}}(f) = \frac{1}{n} \sum_{i=1}^{n} Z_i \longrightarrow R(f) \qquad \text{as } n \to \infty.$$

The condition that f is fixed is essential. An empirical risk minimizer $\widehat{f}_{\mathcal{T}}$ depends on the same observations used to calculate $\widehat{R}_{\mathcal{T}}(\widehat{f}_{\mathcal{T}})$. The simple fixed-function law of large numbers therefore does not justify using training error as an estimate of its future error.

Schematically, the same sample performs two jobs:

$$\mathcal{T} \longmapsto \widehat{f}_{\mathcal{T}} \longmapsto \widehat{R}_{\mathcal{T}}(\widehat{f}_{\mathcal{T}}).$$

Because the predictor was selected to fit that sample, its training error is typically optimistically biased. As model flexibility grows, training error may continue to fall even after test error begins to rise.

0.5.15 Independent Test and Validation Sets

If an independent test set

$$\mathcal{T}_{\text{test}} = \Big((X'_1, Y'_1), \ldots, (X'_q, Y'_q)\Big)$$

is available, the fitted predictor can be evaluated by

$$\widehat{R}_{\text{test}}(\widehat{f}_{\mathcal{T}}) = \frac{1}{q}\sum_{j=1}^{q} r\Big(Y'_j, \widehat{f}_{\mathcal{T}}(X'_j)\Big).$$

Independence matters: the test observations were not used to choose the predictor.

When only one dataset is available, split it into disjoint training and validation subsets,

$$\mathcal{T} = \mathcal{T}_{\text{train}} \,\dot\cup\, \mathcal{T}_{\text{val}}.$$

Fit $\widehat{f}_{\mathcal{T}_{\text{train}}}$ on the training subset and compute

$$\widehat{R}_{\text{val}} = \frac{1}{|\mathcal{T}_{\text{val}}|}\sum_{(X_i,Y_i)\in\mathcal{T}_{\text{val}}} r\Big(Y_i, \widehat{f}_{\mathcal{T}_{\text{train}}}(X_i)\Big).$$

Allocating more observations to training can improve the fitted model, whereas allocating more to validation makes the error estimate more precise.

0.5.16 Cross-Validation

Cross-validation repeats the train–validate pattern so that every observation is held out once and used for training multiple times. For m-fold cross-validation, partition the sample into disjoint folds:

$$\mathcal{T} = \mathcal{T}_1 \,\dot\cup\, \cdots \,\dot\cup\, \mathcal{T}_m.$$

For fold j, define

$$\mathcal{T}^{(-j)} = \mathcal{T} \setminus \mathcal{T}_j, \qquad \widehat{f}^{(-j)} = \widehat{f}_{\mathcal{T}^{(-j)}}.$$

The fold error is

$$\widehat{R}_j = \frac{1}{|\mathcal{T}_j|}\sum_{(X_i,Y_i)\in\mathcal{T}_j} r\Big(Y_i, \widehat{f}^{(-j)}(X_i)\Big),$$

and the cross-validation error is

$$\widehat{R}_{\text{CV}} = \frac{1}{m}\sum_{j=1}^{m} \widehat{R}_j.$$

When $m = n$, each fold contains one observation; this is *leave-one-out cross-validation.*

0.5.16.1 What Cross-Validation Estimates

Condition on the training folds $\mathcal{T}^{(-j)}$. Since the held-out fold $\mathcal{T}_j$ is independent of the data used to fit $\widehat{f}^{(-j)}$,

$$\mathbb{E}\left[\widehat{R}_j \mid \mathcal{T}^{(-j)}\right] = R\Big(\widehat{f}^{(-j)}\Big).$$

Taking expectations once more yields

$$\mathbb{E}[\widehat{R}_j] = \mathbb{E}\left[R\Big(\widehat{f}^{(-j)}\Big)\right].$$

Thus m-fold CV estimates the average risk of predictors trained on $m-1$ folds, not exactly the risk of the final predictor trained on all n observations. Since each fold model uses less data, CV may be slightly pessimistic for the final full-data fit.

0.5.17 Cross-Validation for Hyperparameter Selection

Suppose a family of learning algorithms is indexed by a hyperparameter λ, producing $\widehat{f}_{\mathcal{T},\lambda}$. Examples include a kernel bandwidth, regularization strength, tree depth, or number of nearest neighbors. Compute the CV error for each candidate and select

$$\widehat{\lambda} \in \arg\min_{\lambda} \widehat{R}_{\mathrm{CV}}(\lambda).$$

Then refit on the full training sample using $\widehat{\lambda}$.

Hyperparameter selection changes the complete learning algorithm into the data-dependent procedure

$$\mathcal{T} \longmapsto \widehat{\lambda}(\mathcal{T}) \longmapsto \widehat{f}_{\mathcal{T},\widehat{\lambda}(\mathcal{T})}.$$

Consequently, the CV score minimized to choose $\widehat{\lambda}$ is not a fresh, unbiased test-error estimate for the selected procedure. Honest final evaluation requires data not used in either fitting or hyperparameter selection, or an outer validation loop such as nested cross-validation.